

Projektbericht
Studiengang: Informatik

Erweiterung der Scavenger-Hunt-Webanwendung

von

Fabian Dominik Wottke

3003798

Betreuender Professor: Dr. Marc Hermann

Einreichungsdatum: 15. August 2026

Eidesstattliche Erklärung

Hiermit erkläre ich, **Fabian Dominik Wottke**, dass ich die vorliegenden Angaben in dieser Arbeit wahrheitsgetreu und selbständig verfasst habe.

Weiterhin versichere ich, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben, dass alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und dass die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Ort, Datum

Unterschrift (Student)

Kurzfassung

Im Rahmen dieser Projektarbeit wurde die bestehende Scavenger-Hunt-Webanwendung funktional und gestalterisch erweitert. Die Anwendung ermöglicht es Nutzern, digitale Schnitzeljagden zu erstellen, zu verwalten und zu spielen. Ein besonderer Schwerpunkt der Arbeit lag auf der Verbesserung des Beitrittsprozesses sowie der Einführung einer neuen Möglichkeit zur Antwortauswahl. Darüber hinaus stellte die Optimierung der Benutzeroberfläche und der Benutzererfahrung (UI/UX) auch ein zentrales Ziel der Arbeit.

Als zentrale funktionale Erweiterung wurde ein QR-Code-Scanner entwickelt und in den bestehenden Join-Prozess integriert. Dadurch können Nutzer einer Hunt neben der manuellen Eingabe eines Hunt-Codes auch direkt über einen QR-Code beitreten. Der Scanner verwendet die Browser-Schnittstelle MediaDevices für den Kamerazugriff und die Bibliothek jsQR zur Erkennung und Dekodierung von QR-Codes. Zusätzlich wurden Funktionen wie die Auswahl eines QR-Code-Bildes aus der Galerie, der Wechsel zwischen verfügbaren Kameras sowie eine umfassende Behandlung von Kamera-, Scan- und Eingabefehlern umgesetzt.

Neben der QR-Code-Funktionalität wurde die Benutzeroberfläche der Anwendung überarbeitet und insbesondere für die Nutzung auf mobilen Geräten optimiert. Hierzu wurden zentrale Seiten neu gestaltet, wiederverwendbare UI-Komponenten eingeführt sowie ein einheitliches Farbkonzept mit Light- und Dark-Mode umgesetzt. Darüber hinaus wurde die bestehende Internationalisierung erweitert und eine polnische Benutzeroberfläche ergänzt.

Die umgesetzten Erweiterungen verbessern insbesondere die Benutzerfreundlichkeit beim Beitritt zu einer Schnitzeljagd sowie die Konsistenz und mobile Nutzbarkeit der Anwendung. Die Ergebnisse werden anschließend anhand funktionaler Tests und sowie durch ergänzendes Nutzerfeedback bewertet.

Inhaltsverzeichnis

Eidesstattliche Erklärung	i
Kurzfassung	ii
Abbildungsverzeichnis	v
Tabellenverzeichnis	vi
Abkürzungsverzeichnis	viii
1. Einleitung	1
1.1. Motivation	1
1.2. Problemstellung und -abgrenzung	2
1.3. Ziel der Arbeit	3
1.4. Vorgehen	4
2. Grundlagen	6
2.1. Bestehende Scavenger-Hunt-Anwendung	6
2.2. React	6
2.3. QR-Codes	7
2.4. QR-Code-Erkennung	7
2.5. MediaDevices API	7
2.6. html5-qrcode-Bibliothek	8
2.7. jsQR-Bibliothek	8
2.8. Responsive Webdesign und UI/UX	8
2.9. Zentralisierte Gestaltung und Theme-Verwaltung	9
3. Problemanalyse	10
3.1. Bestandaufnahme	10
3.2. Anforderungen und Featureideen	11
4. Lösungskonzept	13
4.1. Konzept des QR-Code-Scanners	13
4.2. Technologieentscheidung für die QR-Code-Erkennung	14
4.3. Integration in den bestehenden Join-Prozess	15
4.4. Wiederverwendung für QR-Code-basierte Aufgaben	15
4.5. UI/UX-Konzept	16

4.6. Konzept zur Vereinheitlichung des Frontends	17
5. Implementierung	18
5.1. Implementierung des QR-Code-Scanners	18
5.1.1. Kamerazugriff und Kameraverwaltung	20
5.1.2. Verarbeitung des Kamerabildes und QR-Code-Erkennung . . .	21
5.1.3. Scan von QR-Code-Bildern	22
5.1.4. Fehlerbehandlung	24
5.2. Integration in den Join-Prozess	25
5.3. QR-Code-basierte Aufgaben	26
5.4. Überarbeitung der Benutzeroberfläche	28
5.5. Vereinheitlichung der Frontend-Struktur	30
5.6. Internationalisierung	31
6. Inbetriebnahme	32
6.1. Lokale Inbetriebnahme	32
6.2. Vorhandene Server-Deployment-Konfiguration	33
7. User Tests	34
8. Evaluierung	35
8.1. Bewertung der Zielerreichung	35
8.2. Einschränkungen und bekannte Probleme	36
8.3. Gesamtbewertung	38
9. Zusammenfassung und Ausblick	39
9.1. Erreichte Ergebnisse	39
9.2. Ausblick	39
9.2.1. Erweiterbarkeit der Ergebnisse	40
9.2.2. Übertragbarkeit der Ergebnisse	41
10. Verwendete Technologien und Bibliotheken	42
10.1. Frontend	42
10.2. Backend	43
10.3. Datenbank	43
10.4. Containerisierung und Bereitstellung	43
10.5. Relevante Dokumentationsbereiche	44
11. Einsatz von KI-Werkzeugen	45
Literatur	46
A. Anhang A	48
B. Anhang B	50

Abbildungsverzeichnis

5.1. Benutzeroberfläche des implementierten QR-Code-Scanners	19
5.2. QR-Code-basierter Antworttyp mit generiertem QR-Code und Download-Funktion	27
5.3. Überarbeitete mobile Darstellung der Hunt-Bearbeitung mit Detail- und Fragenbereich	29

Tabellenverzeichnis

3.1. Priorisierung der Anforderungen und Featureideen	11
---	----

Listings

5.1. Starten des Kamerastreams mit der MediaDevices API	20
---	----

Abkürzungsverzeichnis

1. Einleitung

1.1. Motivation

Digitale Schnitzeljagden bieten die Möglichkeit, reale Orte mit interaktiven und multimedialen Aufgaben zu verbinden. Die bestehende Scavenger-Hunt-Webanwendung der vorherigen Projektarbeit (Sommersemester 2025) der Hochschule Aalen ermöglicht es, solche Schnitzeljagden zu erstellen und zu spielen. Dabei können unterschiedliche Aufgabentypen, Antwortmöglichkeiten und Hinweisarten genutzt werden, welche durch Text-, Bild-, Video-, Audio- und GPS-Inhalte unterschieden werden können. Dadurch eignet sich die Anwendung beispielsweise dazu, Orte und Angebote der Hochschule Aalen auf spielerische Weise zu erkunden oder auch für andere Bildungseinrichtungen, wie Museen oder Schulen.

Mit zunehmendem Funktionsumfang steigen jedoch auch die Anforderungen an eine intuitive und mobil nutzbare Benutzeroberfläche, welche zudem auch einen eigenen wiedererkennbaren Stil aufweisen sollte. Insbesondere beim Einstieg in eine Schnitzeljagd sollte der notwendige Interaktionsaufwand möglichst gering gehalten werden. Die bisherige Eingabe eines Hunt-Codes stellt zwar eine funktionale Möglichkeit zum Beitritt dar, jedoch erfordert die manuelle Eingabe eines Codes und kann auf mobilen Endgeräten umständlich sein.

Die Integration eines QR-Scanners bietet hierfür eine direkte und für mobile Geräte geeignete Möglichkeit zum Beitritt zu einer Schnitzeljagd. Ein bereitgestellter QR-Code kann mit der Gerätekamera gescannt werden und die darin enthaltene Information unmittelbar für den Beitrittsprozess verwendet werden. Jedoch wird die Implementierung eines solchen Scanners auch für die Antwortmöglichkeit bei den Aufgaben verwendet, sodass man gezielt einen von der Anwendung generierten QR-Code ausdrucken und verstecken kann und im späteren Verlauf auch gefunden und gescannt werden kann. Neben diesen funktionalen Erweiterungen besteht die Motivation der Arbeit darin, die bestehende Benutzeroberfläche hinsichtlich Konsistenz, Verständlichkeit und responsiver Darstellung weiterzuentwickeln.

Ziel der Erweiterung ist es damit, bestehende Funktionen der Scavenger-Hunt-Anwendung leichter zugänglich zu machen und gleichzeitig eine einheitliche Benutzererfahrung auf Desktop und Mobilgeräten zu schaffen.

1.2. Problemstellung und -abgrenzung

Die Scavenger-Hunt-Anwendung stellte zu Beginn dieser Projektarbeit bereits wesentliche Funktionen zum Erstellen, Verwalten und Spielen digitaler Schnitzeljagden bereit. Im Rahmen der Einarbeitung zeigten sich jedoch sowohl funktionale als auch strukturelle und gestalterische Verbesserungspotenziale.

Ein zentrales Verbesserungspotenzial der bestehenden Scavenger-Hunt-Anwendung bestand bei der Nutzung von QR-Codes. Der Beitritt zu einer Hunt sollte über einen direkt in die Webanwendung integrierten QR-Code-Scanner vereinfacht werden, dieselbe Funktionalität des QR-Code-Scanners sollte auch bei entsprechenden Aufgaben innerhalb einer Hunt wiederverwendet werden. Dadurch können QR-Codes beispielsweise als Antwortmöglichkeit für einzelne Stationen eingesetzt werden.

Neben dieser funktionalen Erweiterung zeigten sich bei der Einarbeitung in das bestehende Projekt auch Verbesserungspotenziale hinsichtlich der Benutzeroberfläche. Die Gestaltung der Anwendung wirkte auf vielen Seiten sehr zurückhaltend und war durch einen hohen Anteil weißer Flächen geprägt. Dadurch entstand teilweise ein visuell wenig differenziertes Erscheinungsbild, welches auch keinen wiedererkennbaren Stil aufwies, welcher die Anwendung leichter erkennbar machen würde. Dieser Eindruck wurde auch im Rahmen einer Besprechung mit dem Betreuer thematisiert, wobei insbesondere der starke Weißanteil der bestehenden Oberfläche als Verbesserungspotenzial identifiziert wurde.

Darüber hinaus war beispielsweise die Bearbeitung einer Hunt teilweise unübersichtlich, da Funktionen und Informationen auf der Seite zunächst gesucht werden mussten. Ziel war daher nicht nur eine farbliche Überarbeitung, sondern auch eine klarere visuelle Hierarchie und eine konsistentere Gestaltung der verschiedenen Seiten, um ein leichteres Verständnis und eine bessere Orientierung für den Benutzer zu garantieren.

Auch die Struktur des Frontend-Codes erschwerte teilweise die Einarbeitung und zukünftige Weiterentwicklung. Obwohl der bestehende Quellcode kommentiert war, befanden sich zahlreiche Seitenelemente und zugehörige Gestaltungsregeln direkt innerhalb einzelner Seiten beziehungsweise deren CSS-Dateien. Wiederkehrende Elemente, beispielsweise Buttons und deren Farbdefinitionen, waren dadurch teilweise mehrfach oder seitenbezogen umgesetzt, obwohl sie an verschiedenen Stellen der Anwendung verwendet wurden. Dies führte zu größeren Dateien und machte es notwendig, für Änderungen zunächst nach den jeweiligen Definitionen zu suchen.

Im Rahmen dieser Arbeit wurde daher ein Teil dieser wiederkehrenden Strukturen zentralisiert. Wiederverwendbare Bedienelemente wurden teilweise in eigene Komponenten ausgelagert, welche sich in den dazugehörigen passenden Ordnern befinden, und zentrale Gestaltungswerte, insbesondere Farben, an einer gemeinsamen

Stelle (App.css) definiert und kommentiert. Dadurch sollen Anpassungen leichter nachvollziehbar sein und die visuelle Konsistenz zwischen den Seiten verbessert werden.

Der Schwerpunkt der Arbeit liegt somit auf der funktionalen Erweiterung um eine wiederverwendbare QR-Code-Scan-Funktionalität, der Verbesserung der Benutzerführung sowie einer gezielten Überarbeitung und Vereinheitlichung der Benutzeroberfläche. Eine vollständige Restrukturierung der gesamten bestehenden Codebasis sowie eine grundlegende Neuentwicklung der Hunt-, Benutzer- oder Spiellogik sind dagegen nicht Bestandteil dieser Arbeit und würden den Rahmen der Arbeit sprengen.

1.3. Ziel der Arbeit

Das Ziel der Arbeit besteht darin, die bestehende Scavenger-Hunt-Webanwendung funktional und gestalterisch weiterzuentwickeln. Der Schwerpunkt liegt dabei auf der Entwicklung und Integration eines wiederverwendbaren QR-Code-Scanners, der sowohl für den Beitritt zu einer Hunt als auch für die Beantwortung von Aufgaben innerhalb einer Hunt verwendet werden kann.

Der Scanner soll direkt im Browser funktionieren und den Zugriff auf die Kamera eines Geräts ermöglichen. Neben der Erkennung von QR-Codes über einen laufenden Kamerastream soll auch das Auslesen eines bereits vorhandenen QR-Code-Bildes aus der Galerie unterstützt werden. Darüber hinaus sollen auch unterschiedliche Kameras eines Geräts ausgewählt sowie typische Fehlerfälle, beispielsweise fehlende Kamera oder fehlende Berechtigungen, verständlich behandelt werden.

Ein weiteres Ziel der Arbeit ist die Verbesserung der Benutzeroberfläche und der Benutzererfahrung. Die Anwendung soll insbesondere auf mobilen Endgeräten übersichtlicher und einfacher bedienbar werden. Hierzu werden ausgewählte Seiten hinsichtlich Struktur, responsiver Darstellung, visueller Hierarchie und einheitlicher Gestaltung überarbeitet um eine bessere Orientierung und ein konsistenteres Erscheinungsbild zu schaffen. Zusätzlich soll das bestehende Farbkonzept vereinheitlicht und um einen konsistenten Light- und Dark-Mode ergänzt werden, welches wiederum auch die visuelle Konsistenz der Anwendung verbessern soll.

Zur Verbesserung der Wartbarkeit sollen außerdem ausgewählte wiederkehrende Bestandteile des Frontends zentralisiert und wiederverwendbar gestaltet werden. Dazu gehören insbesondere gemeinsam verwendete Bedienelemente sowie zentrale Farb- und Gestaltungsdefinitionen, jedoch soll auch verhindert werden, dass eine komplette Restrukturierung des bestehenden Frontend-Codes notwendig ist.

Abschließend sollen die umgesetzten Erweiterungen durch funktionale Tests sowie

durch ergänzende Rückmeldungen von Nutzern überprüft werden. Dabei soll insbesondere betrachtet werden, ob die neuen Funktionen verständlich bedienbar sind und wie die überarbeitete Benutzeroberfläche von Personen wahrgenommen wird, die nicht an der Entwicklung beteiligt waren.

1.4. Vorgehen

Zu Beginn der Projektarbeit wurde vom Betreuer die Erweiterung der bestehenden Scavenger-Hunt-Anwendung um eine QR-Code-Funktionalität als zentrale Aufgabe vorgeschlagen. Daraufhin erfolgte zunächst eine Einarbeitung in die bestehende Anwendung und insbesondere in den vorhandenen Beitrittsprozess zu einer Hunt. Dabei wurde untersucht, wie der bestehende Beitrittsprozess funktioniert, welche Daten übermittelt werden und wie ein QR-Code-Scanner in den bestehenden Prozess integriert werden kann. Parallel dazu wurde eine Recherche zu bestehenden QR-Code-Scanner-Bibliotheken durchgeführt und wie man diese Bibliotheken sinnvoll verwendet und in die bestehende Anwendung integriert.

Auf Grundlage dieser Einarbeitung und Recherche wurden erste Lösungsansätze für die QR-Code-Integration sowie weitere mögliche Verbesserungen der Anwendung entwickelt. Die grundlegenden Ziele und der geplante Umfang der Projektarbeit wurden anschließend mit dem Betreuer abgestimmt. Aus diesen Vorgaben wurden die Anforderungen im nächsten Schritt eigenständig priorisiert und in Must-Have-, Should-Have-, Could-Have- und Nice-To-Have-Anforderungen eingeordnet. Diese Priorisierung diente im weiteren Projektverlauf als grobe Orientierung für die Umsetzung und Abgrenzung der einzelnen Erweiterungen.

Anschließend wurde weiter recherchiert, wie ein QR-Code-Scanner innerhalb einer Webanwendung technisch umgesetzt werden kann. Dabei wurden verschiedene mögliche Ansätze und Bibliotheken betrachtet. Zu Beginn wurde unter anderem die Bibliothek `html5-qrcode` als mögliche Lösung in Betracht gezogen, jedoch war auch die Bibliothek `@zxing/browser` eine mögliche Alternative, die jedoch aufgrund der Komplexität und des höheren Aufwands für die Integration in die bestehende Anwendung zunächst nicht weiter verfolgt wurde. Zudem wurden auch Ansätze für die Überarbeitung der Benutzeroberfläche bedacht, insbesondere die Einführung eines konsistenten Farbkonzepts und die Verbesserung des Designs der wichtigsten Seiten und im weiteren mit dem Betreuer festgelegt.

Im folgenden Schritt wurde ein Prototyp für die QR-Code-Integration mit Hilfe der `html5-qrcode`-Bibliothek entwickelt. Dabei wurde zunächst ein einfacher QR-Code-Scanner implementiert, der die Kamera eines Geräts verwendet und QR-Codes erkennen kann. Jedoch zeigte sich, dass diese Implementierung nicht der erwarteten Niveau und Umfang der Projektarbeit entsprach, sodass im weiteren Verlauf eine weitere Recherche zu alternativen Bibliotheken durchgeführt wurde, wodurch die Bi-

bibliothek jsQR als geeignete Lösung identifiziert wurde. Diese Bibliothek ermöglicht eine direkte Verarbeitung von Kamerastreams und eine bessere, kontrollierbare und flexiblere Integration in die bestehende Anwendung. Daraufhin wurde der alte Prototyp erstmalig als Notlösung betrachtet, falls der Prototyp mit der jsQR-Bibliothek nicht erwartungsgemäß funktioniert.

Daraufhin wurde ein neuer Prototyp mit der jsQR-Bibliothek entwickelt, der die Kamera eines Geräts verwendet und QR-Codes erkennen kann und auch schon die groben Design-Erwartungen an die spätere Integration in die bestehende Anwendung entsprach. Daraufhin wurde eine weitere Abstimmung mit dem Betreuer durchgeführt, bei der die geplante Überarbeitung der Benutzeroberfläche leicht konkretisiert wurden und weitere mögliche Verbesserungen des QR-Codes-Scanners besprochen wurden, wie beispielsweise die Möglichkeit, ein QR-Code-Bild aus der Galerie auszuwählen und zu scannen, mehrere Kameras eines Geräts auszuwählen und die genauere Spezifikation typischer Fehlerfälle, wie fehlende Kamera oder fehlende Berechtigungen, verständlich zu behandeln. Aber auch wurde eine weitere erstmals im Hintergrund vergessene Anforderung besprochen, dass der QR-Code-Scanner auch für die Beantwortung von Aufgaben innerhalb einer Hunt wiederverwendet werden sollten.

Im weiteren Verlauf der Projektarbeit wurde der Prototyp des QR-Code-Scanners weiterentwickelt und um die besprochenen Funktionen erweitert. Dabei wurde auch die Integration der Erweiterung der bestehenden Fehlerfälle und deren Behandlung, aber auch die Integration in die Hunt-Beantwortung priorisiert und umgesetzt. Anschließend wurden auch die anderen Funktionen, wie die Auswahl eines QR-Code-Bildes aus der Galerie und die Auswahl einer Kamera implementiert. Anschließend wurde auch die Benutzeroberfläche der Anwendung überarbeitet, insbesondere die wichtigsten Seiten, wie die Startseite, die Hunt-Übersichtsseite und die Hunt-Bearbeitungsseite. Dabei wurden wiederverwendbare UI-Komponenten eingeführt, zentrale Farbdefinitionen in einer gemeinsamen Datei (App.css) definiert und die bestehende Internationalisierung erweitert, sodass die Anwendung auch für eine polnische Benutzeroberfläche vorbereitet ist.

Nach Abschluss der wesentlichen Implementierungsarbeiten wurden die neuen Funktionen sowie die überarbeitete Benutzeroberfläche funktional getestet. Zusätzlich wurde die Anwendung mehreren Personen aus dem Familien- und Freundeskreis gezeigt, um ergänzende Rückmeldungen zur Bedienbarkeit und zur überarbeiteten Benutzeroberfläche zu erhalten.

Abschließend werden die umgesetzten Ergebnisse mit den zu Beginn definierten Anforderungen verglichen und hinsichtlich ihrer Zielerreichung bewertet. Nicht oder nur teilweise realisierte Anforderungen werden dabei ebenfalls dokumentiert und als mögliche Ansatzpunkte für zukünftige Erweiterungen betrachtet.

2. Grundlagen

2.1. Bestehende Scavenger-Hunt-Anwendung

Die bestehende Scavenger-Hunt-Anwendung ist eine webbasierte Anwendung zum Erstellen, Verwalten und Spielen digitaler Schnitzeljagden. Sie basiert auf React im Frontend, FastAPI im Backend und der PostgreSQL Datenbank zur persistenten Speicherung der Anwendungsdaten. Zusätzlich wird Docker für die containerisierte Bereitstellung der Anwendung verwendet. Die vorliegende Projektarbeit baut auf dieser bestehenden Architektur auf und konzentriert sich hauptsächlich auf die Erweiterungen des React-Frontend. Für weitere Informationen zu der bestehenden Architektur enthält dieses Repository unter `/docs/legacy/SoSe25` die Dokumentation der davorigen Projektarbeit.

2.2. React

React wurde bereits in der vorherigen Projektdokumentation als Grundlage des Frontends beschrieben. Im Rahmen dieser Arbeit sind insbesondere die komponentenbasierte Struktur sowie die React-Hooks von Bedeutung. Wiederverwendbare UI-Bestandteile können als eigene Komponenten ausgelagert und auf mehreren Seiten eingesetzt werden, wodurch redundanter Code reduziert wird und eine einheitliche Gestaltung ermöglicht wird. Für die Zustandsverwaltung innerhalb der Komponenten werden unter anderen `useState`, `useEffect` und `useContext` verwendet. Zusätzlich übernimmt `react-router-dom` die Navigation zwischen den verschiedenen Ansichten der Anwendung. Diese Konzepte bilden unter anderem die Grundlage für die Umsetzung des Qr-Code-Scanners sowie die Überarbeitung und Aufteilung bestehender Benutzeroberflächen.

[[13](#), [12](#)]

2.3. QR-Codes

QR-Codes (Quick Response Codes) sind zweidimensionale Codes, in denen Informationen wie Texte, URLs oder Identifikationswerte gespeichert werden können. Die Daten werden in einem quadratischen Muster aus hellen und dunklen Bereichen codiert und können mithilfe einer Kamera oder eines geeigneten Scanners ausgelesen werden. Dabei enthält dieses Muster charakteristische Positionsmarkierungen in den Ecken, welche die Erkennung und Ausrichtung des Codes ermöglichen. In der Scavenger-Hunt-Anwendung werden QR-Codes verwendet, um beispielsweise Hunt-Codes für die jeweiligen Hunts oder um scannbare Antworten für die Stationen in einer Hunt zu generieren und anschließend für die Spieler zum Scannen.

[2]

2.4. QR-Code-Erkennung

Die Erkennung eines QR-Codes in einer Webanwendung erfolgt typischerweise über Bilddaten, die entweder von einer Gerätekamera oder aus einer vorhandenen Bilddatei stammen. Bei einem kamerabasierten Scanner wird zunächst ein Videostream im Browser bereitgestellt. Aus diesem werden regelmäßig einzelne Bildinformationen entnommen und an eine QR-Code-Erkennung weitergegeben, welche die enthaltenen Daten entnimmt und bei der erfolgreichen Erkennung den gespeicherten Inhalt zurückliefert. Für diesen Prozess werden Browser-APIs wie `MediaDevices` für den Zugriff auf die Kamera und geeignete Bibliotheken wie `jsQR` für die Dekodierung des QR-Codes benötigt.

[1, 14]

2.5. MediaDevices API

Die `MediaDevices` API ist eine Schnittstelle in modernen Webbrowsern für den Zugriff auf Mediengeräte wie Kamera oder auch Mikrofone. Über die Methode `getUserMedia()` kann eine Webanwendung nach Zustimmung des Nutzers einen Stream von der jeweiligen zugestimmten Mediumquelle anfordern. Der zurückgelieferte `MediaStream` kann anschließend beispielsweise in einem Video-Element dargestellt und für die weitere Verarbeitung verwendet werden. Dies ermöglicht jedoch auch die Ermittlung der verfügbaren Medienquellen, sodass auch verschiedene verfügbare Kameras gezielt gefunden und ausgewählt werden können. Für den QR-Code-Scanner der Scavenger-Hunt-Anwendung bildet die `MediaDevices` API die Grundlage für den direkten Zugriff auf die Gerätekamera.

[6, 5, 8, 7]

2.6. html5-qrcode-Bibliothek

html5-qrcode ist eine JavaScript-Bibliothek, welche die Erkennung von QR-Codes und anderen Barcodes innerhalb von Webanwendungen ermöglicht. Sie stellt bereits eine weitgehend fertige Scanner-Funktionalität bereit und übernimmt unter anderen den Zugriff auf die Kamera sowie die Auswertung des Kamerastreams. In Rahmen dieser Projektarbeit wurde die Bibliothek zunächst für einen ersten Prototypen des QR-Code-Scanners verwendet. Dadurch konnte die grundsätzliche Funktionsweise und die Integration in die bestehende Anwendung getestet werden.

[14]

2.7. jsQR-Bibliothek

jsQR ist eine weitere JavaScript-Bibliothek zur Erkennung von QR-Codes in Webanwendungen. Im Gegensatz zu html5-qrcode bietet jsQR eine flexiblere und kontrollierbare Integration, da sie lediglich die Dekodierung von QR-Codes übernimmt und die Verarbeitung des Kamerastreams und die Darstellung der Benutzeroberfläche der Anwendung dem Entwickler überlässt. Das bedeutet jedoch auch, dass der Entwickler selbst für die Implementierung der Kamerasteuerung zuständig ist und dadurch auch Funktionalitäten wie der Kamerazugriff, Bildaufnahme und der Scanablauf unabhängig von der Bibliothek umgesetzt werden müssen. Im weiteren Verlauf der Projektarbeit wurde die jsQR-Bibliothek als Grundlage für die Implementierung des finalen QR-Code-Scanners verwendet und mit der MediaDevices API sowie einem eigenen Scanablauf kombiniert.

[1, 4]

2.8. Responsive Webdesign und UI/UX

Responsive Webdesign bezeichnet die Anpassung einer Benutzeroberfläche an unterschiedliche Bildschirmgrößen und Endgeräte. Dabei werden unter anderem Layout, Abstände, Schriftgrößen und Bedienelemente so gestaltet, dass die Anwendung sowohl auf Desktop- als auch auf Mobilgeräten übersichtlich und gut bedienbar bleibt. Im Rahmen von UI/UX spielen zusätzlich eine klare visuelle Hierarchie, konsistente Gestaltung und eine verständliche Benutzerführung eine wichtige Rolle.

Diese Aspekte waren für die Überarbeitung der Benutzeroberfläche der Scavenger-Hunt-Anwendung besonders relevant, da ein großer Teil der Nutzung auch auf mobilen Endgeräten erfolgt.

[9]

2.9. Zentralisierte Gestaltung und Theme-Verwaltung

Bei größeren Webanwendungen ist es wichtig, wiederkehrende Gestaltungselemente wie Farben, Abstände oder weitere Designparameter zentral zu definieren. In CSS können hierfür beispielsweise Variablen verwendet werden, die anschließend von verschiedenen Komponenten und Seiten gemeinsam genutzt werden. Dadurch lassen sich Änderungen an der Gestaltung an einer zentralen Stelle vornehmen und ein einheitliches Erscheinungsbild sicherstellen, jedoch auch die Wartbarkeit des Codes verbessern. Dieses Prinzip wurde in der Scavenger-Hunt-Anwendung unter anderen für die Vereinheitlichung des Farbkonzepts sowie für die Umsetzung von Light- und Dark-Mode verwendet.

[10]

3. Problemanalyse

3.1. Bestandsaufnahme

Zu Beginn der Projektarbeit wurde die bestehende Scavenger-Hunt-Anwendung zunächst der für die geplanten Erweiterungen relevanten Funktionen untersucht. Ein besonderer Fokus lag dabei auf dem Beitrittsprozess einer Hunt, da die Integration eines QR-Code-Scanners als zentrale funktionale Erweiterung vorgesehen war. Dabei ermöglichte der bestehende Ablauf bereits den Beitritt eines Hunts über einen Hunt-Code, jedoch war die direkte Erfassung eines QR-Codes über die Kamera des verwendeten Endgeräts mit der entsprechenden integrierten Scanner-Funktion noch nicht vorhanden. Daher musste zunächst untersucht werden, wie der bestehende Join-Prozess aufgebaut ist, welche Informationen beim Beitritt verarbeitet werden und wie die neue Join-Option, welche die QR-Erkennung verwendet, in den bestehenden Prozess integriert werden kann.

Neben dem Beitrittsprozess wurde auch die bestehende Benutzeroberfläche analysiert, dies brachte hervor, dass die Anwendung funktional bereits einen großen Umfang besaß, jedoch die Gestaltung der Benutzeroberfläche teilweise wenig strukturiert, aber auch visuell zurückhaltend wirkte. Insbesondere der hohe Anteil weißer Flächen führte dazu, dass sich einzelne Bereiche nur gering voneinander abgrenzten. Auf umfangreichen Seiten, beispielsweise der Hunt-Bearbeitungsseite, mussten bestimmte Informationen und Funktionen teilweise gesucht werden, da eine eindeutige visuelle Hierarchie fehlte. Auch auf die Nutzung auf mobilen Endgeräten bestand bei mehreren Ansichten Verbesserungspotenzial.

Darüber hinaus wurde die Struktur des bestehenden Frontend-Codes betrachtet und festgestellt, dass wiederkehrende Bedienelemente und Gestaltungsregeln teilweise direkt innerhalb einzelner Seiten beziehungsweise in deren zugehörigen CSS-Dateien definiert waren. Dadurch existierten ähnliche Elemente und Farbdefinitionen an mehreren Stellen innerhalb des Projekts, wodurch bei Änderungen daher erst ermittelt werden musste, in welcher Datei die jeweilige Gestaltung definiert war. Größere Seiten waren dadurch auch teilweise unübersichtlich und erschwerten die Einarbeitung in die bestehende Codebasis, weil diese innerhalb einer Datei mehrere Aufgaben übernehmen mussten.

Aus der Bestandsaufnahme ergaben sich somit drei zentrale Bereiche für die wei-

tere Projektarbeit: die Erweiterung des bestehenden Beitrittsprozesses um eine kamerabasierte QR-Code-Erkennung, die Überarbeitung der Benutzeroberfläche hinsichtlich Konsistenz, Verständlichkeit und Strukturierung wiederkehrender Bestandteile des Frontends.

3.2. Anforderungen und Featureideen

Die Anforderungen an die Projektarbeit wurden zu Beginn auf Grundlage der bestehenden Anwendung sowie in Abstimmung mit dem Betreuer festgelegt. Anschließend wurden die einzelnen Anforderungen entsprechend ihrer Wichtigkeit für die Projektarbeit priorisiert. Dabei entstanden verschiedene Anforderungen, welche in Must-Have-, Should-Have- und Could-Have-Anforderungen unterteilt wurden, außerdem wurden auch Nice-to-Have-Ideen betrachtet, welche jedoch nicht Bestandteil der priorisierten Umsetzung waren.

Tabelle 3.1.: Priorisierung der Anforderungen und Featureideen

Must-Have
• Hunts sollen neben der manuellen Eingabe eines Hunt-Codes auch über einen QR-Code-Scan beitreterbar sein. • Ein erfolgreicher Scan soll den Nutzer direkt zur zugehörigen Hunt beziehungsweise zum bestehenden Join-Prozess weiterleiten. • Ungültige oder nicht verfügbare QR-Codes sollen zuverlässig erkannt und verständlich behandelt werden. • Der Share- und Publish-Prozess soll verständlich und konsistent aufgebaut sein. • Zentrale Seiten der Anwendung sollen visuell überarbeitet und insbesondere für mobile Geräte besser nutzbar sein.
Should-Have

- Verbesserung des Share-Dialogs durch QR-Code, Link und Kopierfunktion
- Klare Rückmeldungen bei erfolgreichen Aktionen, Fehlern und Ladezuständen
- Konsistente Gestaltung der Join-, Hunt-Detail- und Play-Seite
- Verbesserung der Benutzerführung beim Beitritt zu einer Hunt
- Verständliche Hinweise bei fehlender Berechtigung für den Kamerazugriff

Could-Have

- Verwendung von QR-Codes für einzelne Stationen oder Checkpoints innerhalb einer Hunt
- Bereitstellung alternativer Scan-Möglichkeiten
- Ergänzung von Fortschrittsanzeigen und weiterem visuellen Feedback
- Zusätzliche Verfeinerungen der Benutzeroberfläche
- Kleinere Komfortfunktionen im Bereich Sharing und Hunt-Beitritt

Nice-to-Have

- Mikroanimationen innerhalb der Benutzeroberfläche
- Unterstützung zusätzlicher Design-Themes
- Download von generierten QR-Codes
- Optisch erweiterte Share Karten
- Zusätzliche Gamification-Elemente

4. Lösungskonzept

Auf der Basis der zuvor durchgeführten Problemanalyse und der daraus abgeleiteten Anforderungen wird in diesem Kapitel für die geplanten Erweiterungen der Scavenger-Hunt-Anwendung beschrieben, dabei wird zunächst die grundlegende Funktionsweise des QR-Scanners betrachtet und die Auswahl der eingesetzten Technologien begründet. Anschließend wird die Integration des QR-Code-Scanners beim Beitritt einer Hunt beschrieben und im weiteren Verlauf auch die Wiederverwendung des Scanners für die Beantwortung von Aufgaben innerhalb einer Hunt erläutert. Neben der funktionalen Erweiterung wird außerdem noch die grob geplante Überarbeitung der Benutzeroberfläche beschrieben sowie die Vereinheitlichung wiederkehrende Gestaltungs- und Frontend-Strukturen. Das Lösungskonzept bildet damit die Grundlage für die anschließende technische Umsetzung der einzelnen Erweiterungen.

4.1. Konzept des QR-Code-Scanners

Für die Umsetzung des QR-Code-Scanners wurde zunächst der grundlegende Gedanke betrachtet, wie ein QR-Code-Scanner innerhalb einer Webanwendung umgesetzt werden soll. Dabei wurde die Idee bevorzugt, dass es ein modularer Scanner sein sollte, der unabhängig von einer Seite der Anwendung verwendet werden kann. Dies erleichtert die spätere Integration in die bestehende UI und ermöglicht zudem auch, dass in späteren Abschnitten dieser Anwendung der Scanner leicht wiederverwendet werden kann.

Der Scanner sollte sowohl QR-Codes über die Kamera eines Endgeräts als auch aus vorhandenen Bilddateien erkennen können, zusätzlich sollte der Wechsel zwischen mehreren der Wechsel zwischen mehreren Kameras eines Geräts ermöglicht werden. Außerdem sollte eine verständliche Behandlung typischer Fehlerfälle umgesetzt werden, beispielsweise wenn keine Kamera verfügbar ist.

Ein weiterer wichtiger Bestandteil des Konzepts war die Trennung zwischen der eigentlichen Scan-Funktionalität und der anschließenden Verarbeitung des erkannten Wertes. Dadurch kann derselbe Scanner sowohl für den Beitritt zu einer Hunt als auch für QR-Code-basierte Aufgaben innerhalb einer laufenden Hunt verwendet werden oder sogar in zukünftigen Erweiterungen der Anwendung komplett aus-

getauscht werden, ohne dass die Verarbeitung des erkannten Wertes angepasst werden muss.

Die genauere technische Umsetzung des QR-Code-Scanners wird in den folgenden Abschnitten beschrieben.

4.2. Technologieentscheidung für die QR-Code-Erkennung

Für die Umsetzung des QR-Code-Scanners wurden zunächst verschiedene JavaScript-Bibliotheken betrachtet, unter anderem die Bibliothek `html5-qrcode`. Als erster Ansatz wurde letztendlich die Bibliothek `html5-qrcode` eingesetzt, da sie bereits eine weitgehend vollständige Scanner-Funktionalität bereitstellt und dadurch eine schnelle Entwicklung eines ersten Prototyps ermöglichte. Mit diesem ersten Prototypen konnte der Kamerazugriff wie auch andere grundlegende Funktionalitäten der QR-Code-Erkennung innerhalb der React-Anwendung erfolgreich getestet werden.

Im weiteren Projektverlauf zeigte sich jedoch, dass für die geplanten Erweiterungen eine stärkere und besser kontrollierbare Integration der QR-Code-Erkennung hinsichtlich der Gestaltung der Benutzeroberfläche, der Steuerung des Kamerazugriffs und der Verarbeitung der einzelnen Videobilder erforderlich war, vor allem sodass die geplante Galerie-Funktionalität und die Auswahl zwischen mehreren Kameras eines Geräts kontrollierter umgesetzt werden konnte. Daher wurde im Laufe der Projektarbeit eine bessere Alternative gesucht, wodurch die Bibliothek `jsQR` als weitere Möglichkeit untersucht wurde. Im Gegensatz zu `html5-qrcode` übernimmt `jsQR` ausschließlich die Dekodierung bereitgestellter Bilddaten. Durch die Bibliothek `jsQR` kann die Steuerung der Kamera und die Verarbeitung der einzelnen Videobilder unabhängig von der Bibliothek umgesetzt werden.

Aufgrund dieser höheren Flexibilität wurde für die weitere Entwicklung des QR-Code-Scanners die Bibliothek `jsQR` benutzt, jedoch bedeutete dies auch, dass eine Kombination mit der MediaDevices API gewählt werden musste, um den Zugriff auf die Kamera eines Geräts zu ermöglichen. Durch diesen genannten Wechsel konnten die davor genannten Funktionen wie der Wechsel der Kamera und das Einlesen von QR-Codes aus Bilddateien aber auch eine eigene Fehlerbehandlung gezielter in den Scanner integriert werden. Dies bedeutete auch, dass der zuvor entwickelte Prototyp mit der Bibliothek `html5-qrcode` nicht nur als Machbarkeitsnachweis für die grundsätzliche QR-Code-Integration diente und als Grundlage für die spätere Umsetzung des Scanners, sondern auch als notwendige Absicherung, falls die Umsetzung mit der Bibliothek `jsQR` nicht erwartungsgemäß funktioniert hätte.

4.3. Integration in den bestehenden Join-Prozess

Der QR-Code-Scanner wurde als zusätzliche Eingabemöglichkeit für den bestehenden Beitrittsprozess zu einer Hunt konzipiert, sodass die vorhandene Logik aber auch die bestehende UI weiterhin verständlich aufgebaut sind. Dies bedeutet, dass die vorhandene Logik für den Beitritt zu einer Hunt weiterhin erhalten bleibt und lediglich um die Möglichkeit ergänzt wird, den benötigten Hunt-Code automatisch aus einem QR-Code zu übernehmen.

Nach dem Öffnen des Scanners erfasst der Nutzer den bereitgestellten QR-Code mit der Kamera seines Endgeräts, wodurch ein gültiger Hunt-Code erkannt wird und an den bestehenden Beitrittsprozess übergeben wird. Die anschließende Verarbeitung des Wertes erfolgt dabei auf dieselbe Weise wie bei einem manuell eingegebenen Hunt-Code.

Durch diese Integration des Scanners wird vermieden, dass die bestehende Funktionalität für den Beitritt eines Jagdes doppelt umgesetzt werden muss und zudem auch dadurch entstehende Fehlerquellen vermieden werden. Gleichzeitig bleibt die manuelle Eingabe weiterhin als alternative Möglichkeit erhalten, was insbesondere dann relevant ist, falls keine Kamera verfügbar ist, der QR-Code nicht als Bilddatei verfügbar ist für den Nutzer oder nur als Hunt-Code bereitgestellt wird.

4.4. Wiederverwendung für QR-Code-basierte Aufgaben

Neben dem Beitritt einer Schnitzeljagd sollte auch der QR-Code-Scanner innerhalb einer laufenden Hunt für die bearbeitung von Aufgaben wiederverwendet werden. Dafür wurde vorgesehen, den Scanner unabhängig vom konkreten Anwendungsfall aufzubauen und der Erkannte Scan-Wert an die jeweils aufgerufene Komponente zurückzugeben.

Bei einer QR-Code-basierten Aufgabe kann durch diese Funktionalität der Nutzer den zugehörigen QR-Code direkt mit seiner Kamera scannen und durch dieses Vorgehen die Antwort auf die Aufgabe übermitteln. Der ausgelesene Wert wird anschließend mit der für die Aufgabe hinterlegten Lösung verglichen und kann auf diese Weise in den bestehenden Spielablauf integriert werden.

Durch die Wiederverwendung desselben QR-Code-Scanner muss die Kamera- und QR-Code-Erkennung nicht für unterschiedliche Bereiche der Anwendung mehrfach implementiert werden. Gleichzeitig können Erweiterungen und Fehlerbehandlungen zentral umgesetzt werden, was für eine leichtere Wartbarkeit und eine konsistentere Benutzererfahrung sorgt.

Für die Erstellung entsprechender Aufgaben wurde außerdem vorgesehen, dass

der benötigte QR-Code bereitgestellt beziehungsweise herunterladbar erstellt wird, sodass der Hunt-Ersteller diesen auch an Stationen der Scavenger-Hunt platzieren kann.

4.5. UI/UX-Konzept

Neben der funktionalen Erweiterung um die QR-Code-Integration für den Beitritt aber auch für die Beantwortung von Aufgaben stellte die Überarbeitung der Benutzeroberfläche einen weiteren Schwerpunkt der Projektarbeit dar. Aus der Analyse der bestehenden Anwendung ergab sich insbesondere Verbesserungspotenzial hinsichtlich des visuellen Aussehens, der Übersichtlichkeit einzelner Seiten sowie der Nutzung auf mobilen Endgeräten, zu dem war erstmalig auch die Verbesserung der Nutzung auf Endgeräten mit größeren Bildschirmen, wie beispielsweise Laptops oder Desktop-PCs geplant, jedoch wurde dies im weiteren Verlauf der Projektarbeit nicht weiter verfolgt, da nicht nur die Nutzung auf mobilen Geräten priorisiert wurde, sondern auch nicht genügend eingeplante Zeit für die Umsetzung vorhanden war.

Ein zentrales Ziel bestand darin, die zusammengehörigen Inhalte visuell stärker zu gruppieren, Inhalte bessern für ein angenehmeres Erscheinungsbild strukturieren, wichtige Aktionen deutlicher hervorzuheben und die Anwendung wiedererkennbarer zu gestalten, da dies durch die bisherige sehr blasse Gestaltung nicht gegeben war. Besonders umfangreiche Seiten, wie die Übersicht und Bearbeitung einer Hunt, sollten so strukturiert werden, sodass relevante Informationen schneller und leichter erkennbar sind. Dabei wurde darauf geachtet, wiederkehrende Bedienelemente wie beispielsweise Buttons der Modalfenster möglichst einheitlich darzustellen und deren Position und Funktion über verschiedene Seiten hinweg konsistent zu halten.

Zusätzlich wurde die responsive Darstellung stärker berücksichtigt, dies wurde insbesondere durch die Verwendung von flexiblen Layouts, prozentualen Größenangaben und Media-Queries umgesetzt, sodass Abstände, Größen und Anordnung einzelner Elemente sich an verschiedene Bildschirmgrößen von mobilen Geräten bis hin zu Desktop-PCs anpassen. Jedoch wurde dies für die Nutzung von mobilen Geräten priorisiert, da die Verwendung an diesen Geräten am häufigsten vorkommen. Zudem ist es auch für die Nutzung des QR-Code-Scanners relevant, da dieser häufig direkt auf einem Smartphone verwendet werden.

Für die visuelle Gestaltung wurde außerdem ein einheitlicheres Farbkonzept vorgesehen, welches eine klarere Abgrenzung von Hintergrund-, Inhalts- und Bedienelementen ermöglicht, aber auch die visuelle Darstellung einer Schatzkarte im Light-Modus erzeugt, durch eine beige-braune Farbwahl, sowie einen Dark-Mode, welcher eine dunkel-bräunliche Farbwahl verwendet, sodass die Hunts eine optisch ansprechendere Darstellung für eine Schnitzeljagd erhalten. Dadurch entsteht eine

konsistentere Benutzeroberfläche, die sich an unterschiedliche Nutzungssituationen und Präferenzen anpassen, aber auch eine Farbliche Identität für die Anwendung erzeugen lässt.

Als kleine Erweiterung wurde außerdem die Internationalisierung der Anwendung um eine polnische Benutzeroberfläche kurzfristig vorgesehen, da die Anwendung auch für polnische Nutzer und auch für internationale Studenten der Hochschule Aalen interessant sein könnte

4.6. Konzept zur Vereinheitlichung des Frontends

Neben der visuellen Überarbeitung der Benutzeroberfläche wurde auch die Struktur des Frontends betrachtet, da diese teilweise unübersichtlich war und dadurch die Einarbeitung in die bestehende Codebasis erschwerte. Insbesondere wiederkehrende Bedienelemente und Gestaltungsdefinitionen waren teilweise mehrfach innerhalb verschiedener Seiten hinterlegt oder auch zentriert in einer Datei für eine Seite hinterlegt, welche dadurch größer und unübersichtlicher wurde, aber auch für andere Seiten wiederverwendete Variablen beinhalten musste. Dadurch entstand das Ziel einige wiederkehrende Strukturen stärker zu zentralisieren und wiederverwendbar zu gestalten.

Wiederverwendbare Strukturen wie Buttons, Modale oder andere wiederkehrende UI-Bestandteile sollten deshalb möglichst als eigene Komponenten umgesetzt werden um leichter wiederverwendet werden zu können. Dadurch können sie auf mehreren Seiten verwendet werden, ohne dass identische Strukturen mehrfach oder in verschiedenen Hauptdateien implementiert werden müssen. Dies erzeugt auch das größere Seiten an Struktur und Übersichtlichkeit gewinnen.

Auch die Gestaltungswerte sollten stärker zentralisiert werden, sodass diese nicht mehr in mehreren unterschiedlichen CSS-Dateien definiert werden müssen und so auch leichter angepasst werden können. Dieses Konzept der Zentralisierung der Farben bildet gleichzeitig auch die Grundlage für eine konsistente Umsetzung von Light- und Dark-Mode.

Die geplanten Anpassungen stellen dabei keine vollständige Restrukturierung des bestehenden Frontends dar, stattdessen konzentriert sich die Überarbeitung gezielt auf Bereiche, in denen eine Zentralisierung und Wiederverwendung den größten Nutzen für Übersichtlichkeit, Wartbarkeit und eine Benutzeroberfläche bietet.

5. Implementierung

In diesem Kapitel wird die konkrete Implementierung des im Kapitel 4 entwickelten Lösungskonzepts beschrieben. Hierbei wird auf die konkret verwendeten Entwicklungswerkzeuge etc. Bezug genommen.

Dabei werden insbesondere die Implementierung des Zugriffs auf die Gerätekamera, die Verarbeitung der Kamerabilder, die QR-Code-Erkennung mit `jsQR` sowie die Behandlung unterschiedlicher Fehlerfälle erläutert. Anschließend wird beschrieben, wie die Einbindung des Scanners innerhalb des Beitrittsprozesses einer Hunt sowie für die Beantwortung der Hunt-Aufgaben umgesetzt wurde.

Zudem werden auch die im Rahmen der Projektarbeit vorgenommenen Änderungen an der Benutzeroberfläche sowie die Vereinheitlichung ausgewählter Strukturen beschrieben, darunter gehören unter anderem die responsive Überarbeitung der Seiten, die Umsetzung von Light- und Dark-Mode aber auch die Einführung wiederverwendbarer UI-Komponenten und zentraler Gestaltungsdefinitionen.

5.1. Implementierung des QR-Code-Scanners

Der QR-Code-Scanner wurde in mehrere voneinander getrennte Bestandteile aufgeteilt, wodurch die alte Legacy-QR-Code-Scanner-Implementierung schrittweise durch die neue Implementierung ersetzt werden konnte. Die Verwaltung des Mediastreams erfolgt über eine eigene React-Hook `useCameraStream`, während die Verarbeitung der einzelnen Kamerabilder und die QR-Code-Erkennung durch `useQrScanLoop` übernommen werden und durch den `QrScannerModal` wird letzten Endes die Benutzeroberfläche des Scanners mit den beiden Hooks verbunden. Durch diese Aufteilung dieser Elemente können Kamerasteuerung, Scanlogik und Darstellung unabhängig voneinander bearbeitet und wiederverwendet werden.

Die Benutzeroberfläche des implementierten QR-Code-Scanners ist in Abbildung 5.1 dargestellt.



Abbildung 5.1.: Benutzeroberfläche des implementierten QR-Code-Scanners

5.1.1. Kamerazugriff und Kameraverwaltung

Der Zugriff auf die Kamera eines Geräts erfolgt über die `MediaDevices` API, welche im Hook `useCameraStream` gekapselt wurde, sodass zum starten einer Kamera lediglich ein `MediaStream` über `navigator.mediaDevices.getUserMedia()` angefordert werden muss. Dabei wird automatisch die rückseitige Kamera des Geräts bevorzugt, sofern keine bestimmte Kamera ausgewählt wurde. Zusätzlich wird eine Auflösung von 1280×720 Pixeln für den Stream bevorzugt. Im Fall, dass eine bestimmte Kamera ausgewählt wurde, wird deren `deviceId` direkt als Constraint verwendet.

Im Anschluss wird der zurückgegebene `MediaStream` dem `srcObject` des Videoelements zugewiesen und gestartet, wodurch das aktuelle Kamerabild innerhalb des Scanners bereitgestellt wird.

```
1 const constraints = createCameraConstraints(deviceId);
2
3 const newStream =
4   await navigator.mediaDevices.getUserMedia(constraints);
5
6 streamRef.current = newStream;
7 videoElement.srcObject = newStream;
8
9 await videoElement.play();
```

Listing 5.1: Starten des Kamerastreams mit der `MediaDevices` API

Neben dem Starten der Kamera übernimmt der Hook auch die Verwaltung mehrerer verfügbarer Kameras, welche essentiell für die Auswahl einer bestimmten Kamera innerhalb des Scanners ist und dadurch den Kamerawechsel ermöglicht. Über `navigator.mediaDevices.enumerateDevices()` werden die zur Verfügung stehenden Mediengeräte ermittelt und anschließend nach Geräten des Typs `videoinput` gefiltert, weil nur dieser Typ von den verschiedenen Mediengeräten für die Kamerasteuerung relevant ist. Die durch die Filterung gefundenen Kameras werden anschließend in einem State gespeichert und können über ihre jeweilige `deviceId` ausgewählt werden. Bei einem Wechsel der Kamera wird zunächst der bisherige Stream beendet, sodass die Kamera nicht unnötig weiter aktiv bleibt, anschließend wird mit der neuen ausgewählten Geräte-ID ein neuer Stream gestartet und das Videoelement aktualisiert.

Beim Schließen des Scanners oder beim Wechsel der Kamera müssen die Ressourcen des vorherigen Streams freigegeben werden, um dies zu erreichen, werden alle enthaltenen `MediaStreamTrack`-Objekte über `stop()` beendet und die Verbindung zwischen Stream und Videoelement wieder entfernt. Dies dient dazu, dass die Kamera nach dem Verlassen des Scanners nicht unnötig Ressourcen verbraucht.

Vor dem Start des Kamerastreams muss zunächst geprüft werden, ob die Anwendung in einem sicheren Browserkontext ausgeführt wird, da der Zugriff auf die Kamera nur erfolgen kann, wenn dies gegeben ist. Außerdem wird zusätzlich geprüft, ob die `getUserMedia()`-Methode vom verwendeten Browser unterstützt wird, da es in seltenen Fällen vorkommen kann, dass die MediaDevices API nicht verfügbar ist und somit auch kein Zugriff auf die Kamera bestehen kann. Wenn dieser Fall eintreten sollte, wird die entsprechende Fehlermeldung aufgerufen, dieser Kamera-Fehler wie auch weitere mögliche Fehlerfälle werden in Abschnitt 5.1.4 beschrieben.

5.1.2. Verarbeitung des Kamerabildes und QR-Code-Erkennung

Die eigentliche Erkennung eines QR-Codes erfolgt innerhalb des Hooks `useQrScanLoop`, welcher die vom Videoelement bereitgestellten Kamerabilder verarbeitet und anschließend die Pixeldaten an die Bibliothek `jsQR` übergibt.

Um die Verarbeitung der einzelnen Videobilder zu starten, wird zunächst das Video-/Canvas-Element überprüft, ob diese verfügbar sind und ob diese bereits gültige Bilddaten des Videostreams vorweisen können. Im Anschluss wird die Größe des Canvas-Elements an die aktuelle Auflösung des Videostreams angepasst, sodass die Pixeldaten des Canvas die gleiche Auflösung wie das aktuelle Kamerabild haben. Im Folgenden wird das aktuelle Bild des Videoelements mithilfe von der Funktion `drawImage()` auf das Canvas übertragen.

Durch die Funktion `getImageData()` werden anschließend die Pixeldaten des Canvas ausgelesen, diese Daten enthalten nicht nur die einzelnen Farbwerte, sondern auch die Breite und Höhe des Bildes, sodass diese Werte direkt an die Bibliothek `jsQR` übergeben werden können.

```
context.drawImage(  
  videoElement,  
  0,  
  0,  
  canvasElement.width,  
  canvasElement.height  
);  
  
const imageData = context.getImageData(  
  0,  
  0,  
  canvasElement.width,  
  canvasElement.height  
);
```

```
const qrResult = jsQR(  
  imageData.data,  
  imageData.width,  
  imageData.height  
);
```

Wird innerhalb eines Bildes ein QR-Code erkannt, liefert `jsQR` ein Ergebnisobjekt zurück, im Zuge dessen wird der enthaltene dekodierte Wert ausgelesen und als Scan-Ergebnis innerhalb des Hooks gespeichert, falls hier kein QR-Code erkannt werden konnte, wird der Scanvorgang fortgesetzt bis ein gültiger QR-Code erkannt wurde oder der Scanvorgang abgebrochen wird.

Die kontinuierliche Erkennung wird über `requestAnimationFrame()` gesteuert, dabei wird jedoch nicht jedes dargestellte Videobild an `jsQR` übergeben, sondern über einen Zeitstempel wird geprüft, wann zuletzt ein Scan durchgeführt wurde, sodass die QR-Code-Erkennung in einem gewählten Abstand von 250 Millisekunden erfolgt, dies führt zu einer Reduzierung der Rechenlast und sorgt dafür, dass nicht jedes einzelne Videoelement verarbeitet werden muss. Nach einem erfolglosen Scan wird der nächste Durchlauf erneut über `requestAnimationFrame()` eingeplant.

Sobald ein QR-Code erfolgreich erkannt wurde, wird die nächste Phase des Scanvorgangs gestartet, in dieser Phase wird der erkannte Wert an die übergeordnete Komponente zurückgegeben und der Scanvorgang beendet. Dies geschieht über einen Callback, welcher den erkannten Wert an die übergeordnete Komponente zurückliefert und den Scanvorgang beendet, was jedoch auch dazu führt, dass derselbe QR-Code nicht unmittelbar mehrfach erkannt und weiterverarbeitet wird.

5.1.3. Scan von QR-Code-Bildern

Um die Nutzung des QR-Code-Scanners zu erweitern, wurde die Möglichkeit implementiert, QR-Codes nicht nur über die Gerätekamera zu erkennen, sondern auch durch das Auslesen von QR-Codes ausvorhandenen Bilddateien. Dies ermöglicht es dem Nutzer, einen QR-Code beispielsweise aus einem Screenshot oder einer gespeicherten Bilddatei zu scannen, ohne dass dieser mit der Kamera erfasst werden muss, falls dieser nicht zur Verfügung steht.

Die Verarbeitung einer Bilddatei erfolgt über die Funktion `decodeQrImageFile`, welche die ausgewählte Datei verarbeitet und anschließend die Pixeldaten an `jsQR` übergibt. Die Funktion wird aufgerufen, sobald der Nutzer eine Bilddatei über den Dateiauswahl-Dialog eingereicht hat, nach der besagten Auswahl wird zunächst überprüft, ob die ausgewählte Datei eine gültige Bilddatei ist, bevor diese anschließend über eine temporäre Objekt-URL in ein Bildelement geladen wird und auf ein Canvas gezeichnet wird. Wie im davorigen beschriebenen kamerabasierten Scan

werden im Folge dessen die Pixeldaten mit `getImageData()` ausgelesen und an `jsQR` übergeben, um den QR-Code zu erkennen.

```
const imageData = canvasContext.getImageData(  
    0,  
    0,  
    canvasElement.width,  
    canvasElement.height  
);  
  
const qrResult = jsQR(  
    imageData.data,  
    imageData.width,  
    imageData.height  
);
```

Um auch bei größeren Bilddateien eine effiziente Verarbeitung gewährleisten zu können, wird die maximale Abmessung des zunächst untersuchten Gesamtbildes auf 2000 Pixel begrenzt, dies führt dazu, dass die Bilddatei unter Beibehaltung des Seitenverhältnisses auf eine maximale Breite oder Höhe von 2000 Pixeln verkleinert wird, falls der Fall eintritt, dass die Bilddatei größer ist. Dadurch wird die Rechenlast ein weiteres Mal reduziert und die Verarbeitungsgeschwindigkeit erhöht, ohne dass die Erkennung von QR-Codes innerhalb der Bilddatei beeinträchtigt wird. Zudem wird bei der Dekodierung sowohl die normale als auch die invertierte Darstellung berücksichtigt.

Falls diese erste Dekodierung keinen QR-Code erkennen konnte, wird ein zweiter Erkennungsanlauf gestartet, dieser spezialisiert sich auf die Erkennung von QR-Codes, die innerhalb eines größeren Bildes nur einen vergleichsweise kleinen Bereich einnehmen in dem er die Bilddatei in mehrere quadratische Ausschnitte unterteilt und diese einzeln untersucht. Dabei werden neben einem zentralen Bereich auch Bereiche am oberen, unteren, sowie auch linken und rechten Bildrand betrachtet. Diese Ausschnitte beinhalten jeweils eine Größe von 1000×1000 Pixeln und werden einzeln an `jsQR` übergeben, um die Erkennung von QR-Codes innerhalb der Bilddatei zu verbessern.

Während der Verarbeitung der Bilddatei wird dem Nutzer ein Ladezustand angezeigt, sodass dieser über den aktuellen Status informiert ist, außerdem wird der laufende kamerabasierte Scan vorübergehend angehalten, um die Verarbeitung der Bilddatei nicht zu unterbrechen. Falls ein QR-Code innerhalb eines Bildes erkannt wird, wird der ermittelte Wert über denselben Erfolgsmechanismus wie bei einem Kamerascan weitergegeben, jedoch falls hier bei dem ganzen Prozess kein QR-Code erkannt werden konnte oder ein Fehler auftritt, wird der Nutzer über den Fehler informiert und der kamerabasierte Scan kann anschließend fortgesetzt werden.

5.1.4. Fehlerbehandlung

Da der QR-Code-Scanner auf Browserfunktionen und die Hardware des verwendeten Endgeräts zugreift, können beim Kamerazugriff unterschiedliche Fehler auftreten, welche in unterschiedlicher Weise behandelt werden müssen und für den Nutzer verständlich dargestellt werden sollten. Aus diesem Grund wurde die Fehlerbehandlung direkt in die Kameraverwaltung integriert und nicht ausschließlich innerhalb der Benutzeroberfläche umgesetzt.

Im Hook `useCameraStream` werden auftretende Fehler in ein einheitliches Fehlerobjekt überführt, diese Objekte enthalten neben einer internen Fehlerkennung und einer für den Nutzer vorgesehenen Meldung auch die Information, ob ein erneuter Zugriffsversuch auf die Kamera sinnvoll ist oder auch nicht. Dadurch kann die Benutzeroberfläche unterschiedlich auf vorübergehende und dauerhaft blockierende Fehler reagieren und den Nutzer entsprechend informieren.

Zu den blockierenden Fehlerfälle, welche dafür sorgen dass eine erneute Ausführung des Kamerazugriffs nicht sinnvoll ist, gehören beispielsweise eine verweigerte Kameraberechtigung, eine nicht vorhandene Kamera oder ein nicht unterstützter Kamerazugriff im verwendeten Browser. Zusätzlich wird vor dem Start geprüft, ob die Anwendung in einem sicheren Browserkontext ausgeführt wird, da der Kamerazugriff über `getUserMedia()` einen entsprechenden Kontext voraussetzt.

Andere blockierende Fehler können auch auftreten, jedoch sind diese in die Kategorie der vorübergehenden Fehler einzuordnen, dies bedeutet im Gegensatz zu den blockierenden Fehlern, dass ein erneuter Versuch des Kamerazugriffs sinnvoll ist und der Nutzer die Möglichkeit erhält, dies auch zu tun. Dies ist beispielsweise der Fall, wenn die Kamera momentan nicht gelesen werden kann oder die angeforderten Kameraeigenschaften nicht verfügbar sind, welche durch eine derzeitige Nutzung der Kamera durch eine andere Anwendung verursacht werden kann. Genau in diesen Fällen wird dem Nutzer innerhalb des Scanners die Möglichkeit geboten, ein weiteres Mal den Scanvorgang zu starten, um die Kamera erneut zu initialisieren und den Scanvorgang fortzusetzen, jedoch wird der bisherige Scanvorgang beendet und die zuletzt ausgewählte Kamera erneut gestartet.

Durch die Komponente `QrScannerModal` wird die Fehlerbehandlung innerhalb des Scanners umgesetzt, sodass die Unterscheidung dieser zwei Fehlerkategorien innerhalb der Benutzeroberfläche erfolgt. Bei einem blockierenden Fehler bekommt der Nutzer nur eine entsprechende Fehlermeldung durch die Komponente `CameraUnavailableState` angezeigt, jedoch bei einem vorübergehenden Fehler wird zusätzlich die Möglichkeit geboten, durch die Komponente `CameraRetryOverlay` den Kamerazugriff erneut zu starten und den Scanvorgang fortzusetzen.

Auch die Fehlerbehandlung beim Einlesen von QR-Code-Bilddateien wurde in die Benutzeroberfläche separat behandelt, dies führt dazu, dass der Nutzer bei einem

Fehler innerhalb des Scanners eine verständliche Rückmeldung wie beispielsweise "Kein QR-Code erkannt" oder "Die ausgewählte Datei konnte nicht verarbeitet werden" erhält, anstatt einer technischen Fehlermeldung. Sofern die Kamera weiterhin verfügbar zum Scannen ist, wird der zuvor pausierte Scan anschließend wieder aufgenommen.

Durch diese klar verständliche und differenzierte Fehlerbehandlung wird die Benutzerfreundlichkeit des QR-Code-Scanners verbessert und der Nutzer kann dementsprechend auf die jeweilige Situation reagieren und nicht durch unverständliche Fehlermeldungen verwirrt werden. Zudem bekommt der Nutzer zu der jeweiligen Fehlerkategorie eine passende Möglichkeit, den Scanvorgang fortzusetzen, falls dies möglich ist.

5.2. Integration in den Join-Prozess

Für die Integration des QR-Code-Scanners in den bestehenden Beitrittsprozess wurde die bestehende Join-Seite nur um die Möglichkeit erweitert, sodass man auch über den QR-Code-Scanner mit einem QR-Code, welches einen gültigen Hunt-Code enthält, die entsprechende Hunt beitreten kann. Diese Erweiterung wurde mit einem zusätzlichen Button realisiert, welcher den QR-Code-Scanner-Modal öffnet und den Scanner startet. Daraufhin wird beim erfolgreichen Scan der dekodierte Inhalt über eine Callback-Funktion `onScanSuccess` zurückgegeben.

Die weitere Verarbeitung erfolgte über die bereits bestehende Funktion `joinWithCode`, welche auch bei der manuellen Eingabe eines Hunt-Codes verwendet wird. Dadurch muss die Beitrittslogik nicht doppelt implementiert werden, dies ermöglicht eine konsistente Verarbeitung. Vor dem eigentlichen Beitritt wird der übergebene Wert mit `extractHuntCode` ausgewertet, dies ermöglicht sowohl die Verarbeitung eines direkt angegebenen sechsstelligen Hunt-Codes als auch die Verarbeitung eines QR-Codes. Im ungünstigen Fall, wurde auch eine Rückfalllösung implementiert, welche nach einer sechsstelligen Zahl innerhalb des übergebenen Inhalts sucht, sodass auch ein QR-Code, der nicht direkt einen Hunt-Code enthält, aber eine sechsstellige Zahl beinhaltet, verarbeitet werden kann.

Wurde ein gültiger Hunt-Code ermittelt, wird über die bereits bestehende Backend-Schnittstelle eine `POST`-Anfrage an den entsprechenden Join-Endpunkt gesendet. Bei einer erfolgreichen Antwort wird der Nutzer anschließend zur Startansicht der ausgewählten Hunt weitergeleitet, so wie es auch bei der manuellen Eingabe eines Hunt-Codes der Fall ist. Im Fall, dass kein gültiger Hunt-Code ermittelt werden konnte oder der Beitritt fehlschlägt, wird die vorhandene Fehleranzeige der Join-Seite verwendet, sodass der Nutzer über den Fehler informiert wird.

Der Vorteil dieser Umsetzung liegt darin, dass für den QR-Code-Scanner keine spe-

zielle separate Beitrittslogik entwickelt werden musste, was nicht nur die Implementierung vereinfacht, sondern auch die Wartbarkeit und Konsistenz der Anwendung verbessert. Das heißt auch, dass der Scanner lediglich einen zusätzlichen Eingabeweg bereit stellt, während Validierung, Backend-Kommunikation und Navigation gemeinsam mit der manuellen Eingabe verwendet werden kann.

5.3. QR-Code-basierte Aufgaben

Neben dem Beitritt zu einer Hunt wurde der QR-Code-Scanner auch in den eigentlichen Spielablauf integriert. Hier wurde letztendlich der bestehende Antworttyp einer Aufgabe um die Option `qr_code` erweitert, sodass der Ersteller einer Hunt Aufgaben erstellen kann, die auch die Funktionalität des QR-Code-Scanners nutzen. Bei der Auswahl dieses Antworttyps wird automatisch ein zufälliger Wert erzeugt, der als korrekte Antwort der Aufgabe gespeichert und anschließend als QR-Code dargestellt wird.

Für die Darstellung des QR-Codes in der Bearbeitungsansicht einer Aufgabe wird die Bibliothek `react-qr-code` verwendet. Diese Bibliothek ermöglicht die einfache Erzeugung eines QR-Codes aus einem beliebigen Textwert, sodass der für die Aufgabe generierte Wert direkt in einen QR-Code eingebettet werden kann. Um auch die Nutzung des QR-Codes außerhalb der Anwendung zu ermöglichen, wurde auch eine Download-Funktion implementiert, welche beim Download den QR-Code als SVG-Datei speichert. Dadurch kann der Ersteller einer Hunt den QR-Code beispielsweise ausdrucken und an der für die Aufgabe vorgesehenen Station platzieren, sodass die Nutzer der Anwendung den QR-Code direkt mit ihrer Kamera scannen können, um die Aufgabe zu lösen.

Die Darstellung einer entsprechenden QR-Code-Aufgabe innerhalb der Bearbeitungsansicht ist in Abbildung 5.2 dargestellt.

Während einer laufenden Hunt wird bei Aufgaben des Typs `qr_code` anstelle der gewöhnlichen Texteingabe eine entsprechende Scan-Aufforderung angezeigt. Über die Schaltfläche zum Scannen wird dasselbe `QrScannerModal` geöffnet, welches auch für den Beitrittsprozess verwendet wird. Dadurch muss dieser Scanner nicht erneut implementiert werden, zudem können Kamerazugriff, Bilderkennung, Galerie-Funktion und Fehlerbehandlung ohne eine zweite Scanner-Implementierung wiederverwendet werden.

Nach einer erfolgreichen Erkennung wird der ausgelesene Wert mit der für die aktuelle Aufgabe im Backend gespeicherten Antwort verglichen, falls beide Werte übereinstimmen, wird die Aufgabe über den bestehenden Ablauf als abgeschlossen markiert und die Hunt fortgesetzt. Bei einem nicht passenden QR-Code wird der Nutzer darüber informiert und verbleibt bei der aktuellen Aufgabe.



Abbildung 5.2.: QR-Code-basierter Antworttyp mit generiertem QR-Code und Download-Funktion

Diese weitere Möglichkeit der Benutzung des QR-Code-Scanners ermöglicht, dass die Funktion nicht nur als Einstiegspunkt verwendet werden kann, sondern auch als interaktives Element innerhalb der einzelnen Stationen benutzt werden kann.

5.4. Überarbeitung der Benutzeroberfläche

Parallel zur Implementierung der QR-Code-Funktionalität wurde auch die Benutzeroberfläche der Scavenger-Hunt-Anwendung langsam überarbeitet, um die Benutzerfreundlichkeit zu verbessern und die Anwendung optisch ansprechender zu gestalten. Dabei lag der Fokus insbesondere auf einer klareren visuellen Struktur, aber auch vor allem auf eine Verbesserung der Farbgestaltung, der Typografie und der Nutzung auf mobilen Endgeräten, da diese Nutzung am häufigsten vorkommt, aber auch die Anwendung verblässen lässt und auch die Typografie nicht wirklich zeitgemäß wirkt. Dafür wurden insbesondere die Startseite, die Hunt-Übersicht, die Bearbeitung einer Hunt sowie der Beitritts- und Spielablauf überarbeitet, jedoch wurden die Funktionen der Anwendung nicht verändert, sondern nur ihre Darstellung und Struktur überarbeitet.

Ein wesentlicher Bestandteil der Überarbeitung war die stärkere visuelle Trennung unterschiedlicher Inhaltsbereiche, um Informationen und zugehörige Aktionen deutlicher zu gruppieren und einen Verständlichen Aufbau gewährleisten zu können. Besonders bei umfangreicheren Ansichten, wie beispielsweise der Bearbeitung einer Hunt, wo die Kartenansicht strukturiert wurde und auch die Öffnung der zwei vorhandenen Kartenansichten, erstmal für den Ersteller geschlossen sind, sodass man leichter erkennen kann, welche Funktion verändert werden können.

Zudem wurde die responsive Darstellung der Anwendung weiterentwickelt, für einzelne Seiten wurde ein einheitlicher Seitenbereich mit einheitlichen Abständen eingeführt, dabei werden horizontale Abstände abhängig von der verfügbaren Bildschirmbreite angepasst, sodass die Inhalte sowohl auf größeren als auch auf kleineren Bildschirmen ausreichend Platz erhalten. Auch einzelne Komponenten wie der QR-Code-Scanner wurden so gestaltet, dass sie sich an kleinere Viewports anpassen und auf mobilen Geräten vollständig bedienbar bleiben.

Zusätzlich wurde auch eine weitere Anpassung unternommen, welche fast alle Seiten der Anwendung betrifft, nämlich die Einführung eines oberen Dashboard-Bereichs, welcher das Logo der Hochschule Aalen und zwei Buttons enthält. Der erste Button dient der Information über die Applikation, während der zweite Button für die zukünftige Erweiterung eines Tutorials der einzelnen Seiten vorgesehen ist.

Im Rahmen der Überarbeitung der visuellen Gestaltung wurde auch die Typografie der Anwendung angepasst, um eine modernere und besser Lesbarkeit zu gewährleisten. Dabei wurde die Schriftart *Bricolage Grotesque* eingeführt, welche für alle

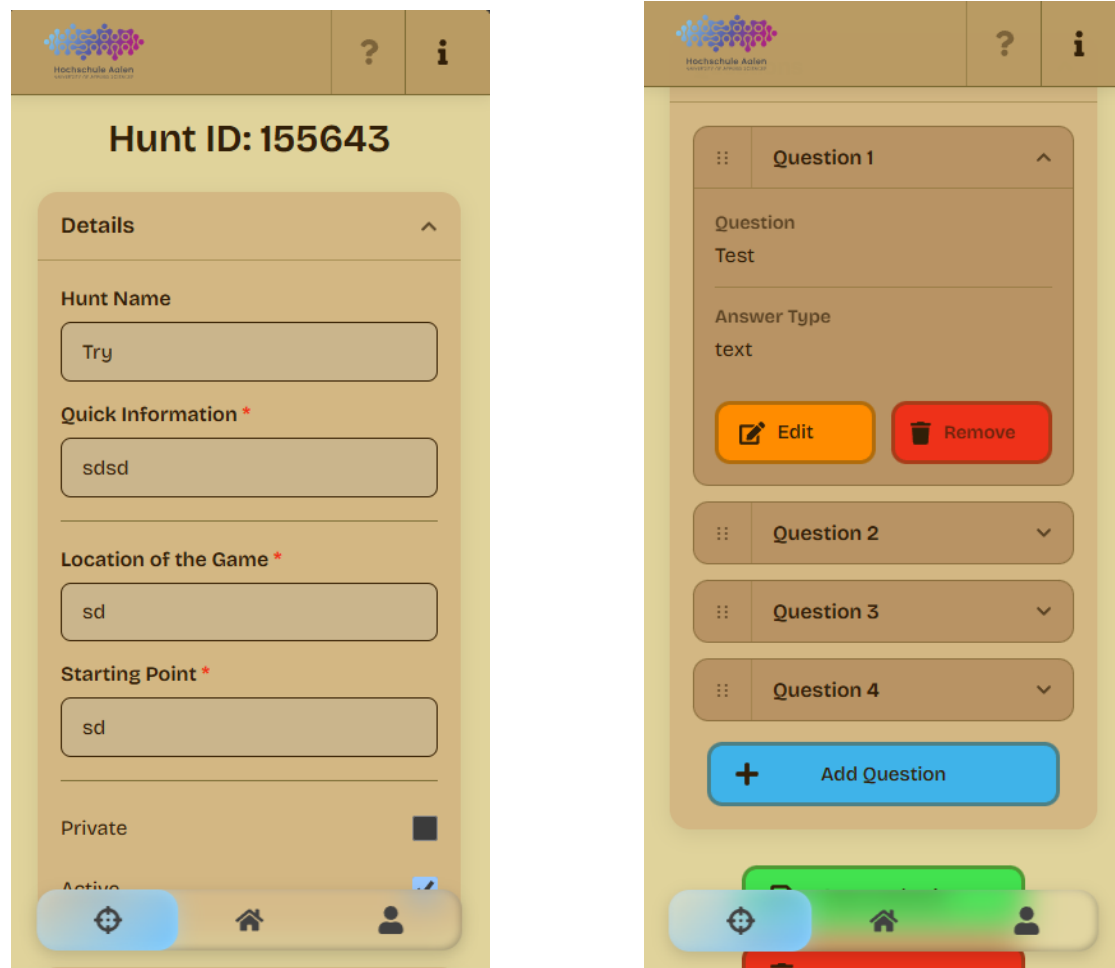


Abbildung 5.3.: Überarbeitete mobile Darstellung der Hunt-Bearbeitung mit Detail- und Fragenbereich

Textelemente der Anwendung verwendet wird und auch dadurch einen besseren Wiedererkennungswert erzeugt, welcher professioneller aber auch zu gleich modern und leicht verspielt wirkt.

Neben dem Layout wurde auch das Farbkonzept überarbeitet und die Darstellung für Light- und Dark-Mode innerhalb der Datei `App.css` vereinheitlicht. Dies ermöglicht eine konsistente Farbgestaltung über die gesamte Anwendung hinweg und sorgt dafür, dass die Farbwahl bei Änderungen zentral angepasst werden können, ohne dass diese an mehreren Stellen der Anwendung angepasst werden müssen.

5.5. Vereinheitlichung der Frontend-Struktur

Im Zuge der Überarbeitung der Benutzeroberfläche wurde letztendlich auch zum Teil die Struktur des Frontends angepasst, um die Wiederverwendbarkeit von Komponenten zu verbessern und die Übersichtlichkeit einzelner Seiten zu erhöhen. Dabei wurden hauptsächlich nur die relevanten Bereiche der Anwendung überarbeitet, welche ich für eine Verbesserung der Benutzeroberfläche benutzt habe, wodurch nicht die ganze Anwendung in dieser Hinsicht überarbeitet wurde. Ziel war es insbesondere, mehrfach verwendete Elemente zentral bereitzustellen und auch umfangreichere Seiten wie auch in anderen Seiten eingebaute Komponenten übersichtlicher aufzubauen.

Ein Beispiel hierfür ist die Einführung von wiederverwendbaren Komponente wie `AppButton` oder auch `AppInput`. Statt Buttons mit ähnlichen Gestaltungsregeln auf verschiedenen Seiten oder auf der schon überfüllten Datei `Home.css` jeweils separat umzusetzen, kann die gemeinsame Komponente mit unterschiedlichen Varianten verwendet werden. Dadurch werden beispielsweise Aktionen wie Speichern, Löschen, Zurückgehen oder Bestätigen über verschiedene Ansichten hinweg einheitlicher dargestellt. Diese Komponente wird unter anderem im Hunt-, Spiel- und Bearbeitungsbereich eingesetzt.

Auch umfangreichere Seiten wurden teilweise in kleinere Komponenten aufgeteilt, wie beispielsweise die Bearbeitung einer Hunt, wo die Bereiche für die allgemeinen Hunt-Informationen und die Verwaltung der Fragen in die Komponenten `EditHuntDetails` und `EditHuntQuestions` ausgelagert wurden. Die übergeordnete Seite `EditHunt` übernimmt weiterhin die zentrale Datenverwaltung und Kommunikation mit dem Backend, jedoch wird die Darstellung einzelner Bereiche stärker voneinander getrennt.

Wie schon vorher erwähnt, wurde auch die Gestaltungswerte der Anwendung in der zentralen Datei `App.css` zusammengeführt. Dort werden unter anderem Farben für Hintergründe, Texte, Oberflächen, Rahmen und Statuszustände über CSS-Variablen definiert. Diese Werte können anschließend von verschiedenen Komponenten ver-

wendet werden, wodurch Änderungen am Farbkonzept nicht mehr an zahlreichen voneinander unabhängigen Stellen vorgenommen wird.

Diese zentralen Definition wird auch für die Umsetzung des Light- und Dark-Mode verwendet, wodurch die Farbwerte des Dark-Modus automatisch von in der zentralen Datei übernommen werden, falls der Dark-Mode aktiviert ist, dadurch sind diese Farbwerte auch nur noch in einer zentralen Stelle definiert.

Die hier genannten Anpassungen reduzieren die Komplexität der einzelnen Seiten und verbessern die Wiederverwendbarkeit von Komponenten, jedoch bleibt die bestehende Grundstruktur der Anwendung immer noch erhalten.

5.6. Internationalisierung

Die Bestehende Scavenger-Hunt-Anwendung wurde ursprünglich nur in deutscher und in englischer Sprache entwickelt, jedoch wurde im Verlauf der Projektarbeit entschieden, die Anwendung um eine polnische Benutzeroberfläche zu erweitern, um auch polnischen Nutzern die Verwendung der Anwendung zu ermöglichen. Diese bestehende Internationalisierung benutzt die Bibliotheken `i18next` und `react-i18next`, dies ermöglicht anstelle fest eingebundener Texte Übersetzungsschlüssel zu verwenden, welche über die Funktion `t()` abhängig von der aktuell ausgewählten Sprache aufgelöst werden.

Im Rahmen der Projektarbeit wurde nur eine zusätzliche Sprache implementiert, jedoch wurde die bestehende Internationalisierung auch um einige weiteren Übersetzungsschlüssel im deutschen und englischen Sprachpaket erweitert, sodass auch neu hinzugefügte Inhalte wie der QR-Code-Scanner, auch in den bestehenden Sprachpaketen übersetzt werden.

Die Auswahl der Sprache erfolgt über die bereits bestehende Sprachverwaltung, welche im Benutzerprofil verfügbar ist. Dort kann der Nutzer zwischen den Sprachen Deutsch, Englisch und Polnisch auswählen, wobei die Auswahl über die Funktion `i18n.changeLanguage()` an das Internationalisierungssystem übergeben wird. Die ausgewählte Sprache wird zusätzlich im lokalen Speicher des Browsers gesichert, sodass die Einstellung auch nach einem erneuten Laden der Anwendung erhalten bleibt.

6. Inbetriebnahme

Die grundlegende Struktur für die lokale Inbetriebnahme und das Deployment der Scavenger-Hunt-Anwendung wurde bereits im vorherigen Projektstand eingerichtet und im Rahmen dieser Projektarbeit weiterverwendet. Dabei stehen weiterhin unterschiedliche Docker-Compose-Konfigurationen für die lokale Entwicklungsumgebung und den Betrieb auf einem Server zur Verfügung.

Die im Rahmen dieser Arbeit vorgenommenen Erweiterungen konnten in die bestehende Deployment-Struktur integriert werden, ohne dass eine grundlegende Änderung der vorhandenen Container-Architektur erforderlich war. Durch diesem Grund wird dieses Kapitel der Inbetriebnahme der Anwendung nur als eine kurze Zusammenfassung der bereits im vorherigen Projektstand beschriebenen Inbetriebnahme und Deployment der Anwendung dienen.

6.1. Lokale Inbetriebnahme

Für die lokale Inbetriebnahme der Anwendung wurde bereits im vorherigen Projektstand eine Docker-Compose-Konfiguration-Datei erstellt, diese Datei enthält die notwendigen Dienste für die lokale Entwicklung für die PostgreSQL-Datenbank, das FastAPI-Backend und das React-Frontend.

Die lokale Inbetriebnahme der Anwendung kann aus dem Hauptverzeichnis des Projekts mit folgendem Befehl gestartet werden:

```
docker compose -f docker-compose-local.yaml up --build
```

Docker erstellt dabei die benötigten Container und startet anschließend die einzelnen Dienste, im folgenden wird das Frontend der Anwendung über `http://localhost:3000` bereitgestellt, während das Backend über Port 8000 erreichbar ist. Dabei wird die Kommunikation zwischen Frontend und Backend über die Umgebungsvariable für die API-Basis konfiguriert.

6.2. Vorhandene Server-Deployment-Konfiguration

Neben der lokalen Inbetriebnahme existiert aus dem vorherigen Projektstand bereits eine Konfiguration für das Deployment auf einem Server, welches als Zielsystem ein UGreen NAS verwendet, auf dem die Anwendung mithilfe von Docker betrieben wurde.

Diese Deployment-Variante wurde im Rahmen dieser Projektarbeit nicht verwendet oder getestet. Die bestehende Konfiguration verwendet Docker Compose und Traefik für die Bereitstellung der Anwendung und den externen Zugriff über HTTPS.

Für ein zukünftiges Deployment des aktuellen Projektstands wären jedoch Anpassungen notwendig. Außerdem enthält die bestehende `docker-compose.yml` noch serverbezogene Werte aus dem vorherigen Projektstand, die vor einer erneuten Bereitstellung angepasst werden müssten.

Ob die bestehende Server-Konfiguration mit dem aktuellen Projektstand vollständig funktionsfähig ist, wurde im Rahmen dieser Arbeit nicht überprüft.

7. User Tests

Gegen Ende der Projektarbeit wurde die Anwendung zusätzlich mehreren Personen aus dem Familien- und Freundeskreis gezeigt, dies heißt auch, dass keine umfangreiche oder standardisierte Nutzerevaluation im Vordergrund stand, sondern vielmehr ein zusätzlicher Eindruck zu erhalten, wie die überarbeitete Benutzeroberfläche von Personen wahrgenommen wird, die nicht an der Entwicklung beteiligt waren.

Die Anwendung wurde dabei auf die Smartphones der jeweiligen Personen bereitgestellt, sodass die Testpersonen die Anwendung auf ihrem eigenen Gerät ausprobieren konnten, jedoch wurde zum Testen der QR-Code-Funktionalität verschiedene Laptops verwendet, da die Testperson durch die lokale Bereitstellung der Anwendung auf dem Laptop die Kamera des Gerätes verwenden konnte.

Während der Testphase wurde die Anwendung von den Testpersonen auf ihre Bedienbarkeit und Verständlichkeit getestet und anschließend wurden die Personen zu ihren Eindrücken befragt. Im Vordergrund stand dabei die Überprüfung der überarbeiteten Benutzeroberfläche aber auch die Probleme bei der Nutzung der überarbeiteten Anwendung, falls diese aufgetreten würden.

Die Rückmeldungen der Testpersonen waren insgesamt positiv, zeigten jedoch auch einzelne kleinere Verbesserungspotenziale. Durch diese Anmerkungen und Hinweise konnten noch kleinere Anpassungen an der Benutzeroberfläche vorgenommen werden um die Bedienbarkeit und Verständlichkeit der Anwendung weiter zu verbessern.

Die User Tests dienten damit vor allem als ergänzende Rückmeldung zum aktuellen Entwicklungsstand und als zusätzliche Möglichkeit, kleinere UI/UX-Probleme vor Abschluss der Arbeit zu erkennen und zu beheben.

8. Evaluierung

In diesem Kapitel werden die im Rahmen der Projektarbeit umgesetzten Erweiterungen abschließend bewertet und dabei werden die gesetzten Ziele betrachtet, wie diese erreicht wurden, welche Einschränkungen weiterhin bestehen und welche Bereiche bei einer zukünftigen Weiterentwicklung noch verbessert werden könnten.

8.1. Bewertung der Zielerreichung

Wie schon in den vorherigen Kapiteln beschrieben, war das zentrale Ziel der Projektarbeit die Erweiterung der bestehenden Scavenger-Hunt-Anwendung um eine wiederverwendbare QR-Code-Funktionalität, welche sowohl für den Beitritt zu einer Hunt als auch für QR-Code-basierte Aufgaben verwendet werden sollte. Dieses Ziel wurde glücklicherweise erreicht auch wenn die erste Umsetzung des QR-Code-Scanners nicht vollständig die geplante Erwartung erfüllte.

Neben der Implementierung des QR-Code-Scanners wurde auch das Einlesen von QR-Codes aus Bilddateien umgesetzt, sodass der Scanner nicht nur über die Gerätekamera, sondern auch über die Auswahl einer Bilddatei genutzt werden kann und auch der Wechsel zwischen Kameras erfolgreich umgesetzt. Hier war die größte Herausforderung, dass die Galeriebilder nicht immer richtig verarbeitet werden konnten, sodass ich mir überlegen musste, wie man dies beheben kann, sodass es zuverlässig in den meisten Fällen funktioniert. Jedoch deswegen musste auch die Behandlung von Fehlerfällen innerhalb des Scanners stark aufgebaut werden, mit dieser Aussage meine ich nicht nur die Fehlermeldungen die der Nutzer bekommt, sondern vor allem die Debugmeldungen, welche man in der Konsole auslesen kann. Dies ermöglichte eine leichte Nachvollziehbarkeit der auftretenden Fehler und eine deutlich organisiertere und strukturierte Fehlerquellenbehandlung, sodass auch bei Komplikationen des Scanners die Fehler schnell gefunden, nachvollgezogen und behoben werden konnten.

Auch die geplante Überarbeitung der Benutzeroberfläche wurde umgesetzt, dabei wurden einige Mängel der alten Benutzeroberfläche behoben, jedoch erfolgte nicht die vollständige Überarbeitung der Benutzeroberfläche, welche ich am Anfang der Projektarbeit geplant hatte. Dies beinhaltet, einen stärkeren Wiedererkennungswert der Anwendung durch Animationen, besondere UI/UX-Elemente, aber auch eine

stärkere Strukturierung der einzelnen Seiten. Dies führte dazu, dass die Benutzeroberfläche zwar verbessert wurde, aber nicht in dem Umfang, wie es ursprünglich geplant war. Dies würde jedoch für eine deutlich größere Überarbeitung der bestehenden Frontend-Struktur erfordern, welche im Rahmen dieser Projektarbeit nicht umgesetzt werden konnte, da die Zeit für eine vollständige Überarbeitung der Frontend-Struktur nicht ausgereicht hätte.

Darüber hinaus wurden Bestandteile der Anwendung, die mehrfach verwendet werden, in wiederverwendbare Komponenten ausgelagert, sodass diese leichter in die Anwendung integriert werden können und auch wenn dies nicht in allen Bereichen der Anwendung umgesetzt wurde und zu dem auch keine konkrete Anforderung der Projektarbeit war, wurde dies dennoch umgesetzt, sodass es für eine zukünftige Weiterentwicklung der Anwendung einfacher ist, diese Elemente zu benutzen.

Ein Vergleich mit den zu Beginn priorisierten Anforderungen zeigt, dass die zentralen Must-Have-Anforderungen weitgehend umgesetzt werden konnten. Der Beitritt zu einer Hunt über einen QR-Code wurde in den bestehenden Join-Prozess integriert, ungültige beziehungsweise nicht verarbeitbare QR-Codes werden entsprechend behandelt und die wichtigsten Ansichten der Anwendung wurden hinsichtlich ihrer mobilen Nutzung und Gestaltung überarbeitet. Auch der bestehende Share- und Publish-Prozess wurde strukturell angepasst, sodass die zugehörigen Funktionen verständlicher und übersichtlicher dargestellt werden.

Auch mehrere Should- und Could-Have-Anforderungen konnten umgesetzt werden. Dazu gehören insbesondere die verständlichere Fehlerbehandlung beim Kamerazugriff, die Verwendung von QR-Codes für Aufgaben innerhalb einer Hunt sowie das Scannen bereits vorhandener QR-Code-Bilder. Andere Anforderungen wurden dagegen nur teilweise oder nicht umgesetzt. Der Download generierter QR-Codes wurde umgesetzt, während die geplante Link- und Kopierfunktion innerhalb des Share-Bereichs sowie zusätzliche Fortschrittsanzeigen und umfangreicheres visuelles Feedback nicht realisiert wurden.

Insgesamt konnte die Projektarbeit trotz einiger Einschränkungen erfolgreich umgesetzt werden und meinen neukalkulierten Erwartungen entsprechen, sodass die wichtigsten Ziele der Projektarbeit erreicht wurden.

8.2. Einschränkungen und bekannte Probleme

Trotz der insgesamt erfolgreichen Umsetzung konnten im Rahmen der Projektarbeit nicht alle ursprünglich geplanten beziehungsweise während der Entwicklung entstandenen Ideen vollständig umgesetzt werden. Hierzu zählt insbesondere die vollständige Überarbeitung der Benutzeroberfläche, welche im Rahmen der Projektarbeit nur teilweise der anfangs gesetzten Erwartungen umgesetzt werden konnte.

Außerdem konnte die Datei des Downloads des QR-Codes nicht in allen Erwartungen umgesetzt werden, da die Darstellung des QR-Codes in der SVG-Datei nicht passend für eine direkte Verwendung ist. Zudem wurden die Mikroanimationen, Design-Themes, Gamification-Elemente und die optisch erweiterte Share-Karte nicht umgesetzt, da dies einen doch deutlicheren größeren Aufwand hätte.

Neben diesen nicht vollständig realisierten Erweiterungen bestehen im aktuellen Stand der Anwendung noch einzelne bekannte Fehler, welche ich während der Projektarbeit identifizieren konnte, jedoch nicht mehr vollständig beheben konnte. Diese betreffen jedoch nicht hauptsächlich die neu implementierte QR-Code-Funktionalität, sondern bereits bestehende beziehungsweise andere Bereiche der Anwendung.

Ein bekanntes Problem betrifft die Bearbeitung von Fragen innerhalb einer Hunt, genauer gesagt die Bearbeitung von Multiple-Choice-Fragen und die Beantwortung von diesem Aufgabentyp in einer Hunt. Dabei kann es vorkommen, dass die Auswahlmöglichkeiten nicht korrekt dargestellt werden, sodass eine Auswahlmöglichkeit angeklickt wurde, jedoch eine andere Auswahlmöglichkeit als ausgewählt angezeigt wird. Dieser Fehler wurde während den letzten Phasen der Projektarbeit identifiziert, wodurch kurzfristig eine Lösung für diesen Fehler nicht mehr genauer erkannt werden konnte und auch somit nicht mehr behoben wurde.

Ein weiterer bekannter Fehler betrifft die Bearbeitung von Aufgaben innerhalb einer Hunt, genauer gesagt das Löschen von Aufgaben. Dabei kann es vorkommen, dass die Löschung einer Aufgabe nicht korrekt durchgeführt wird, sodass die Aufgabe trotzdem weiterhin in der Aufgabenliste angezeigt wird, dies entsteht durch einen HTTP-500-Fehler, welcher beim Löschen der Aufgabe gesendet wird. Da dieser Fehler im Backend beziehungsweise im bestehenden Ablauf der Aufgabenverwaltung auftritt und nicht zu den zentralen Aufgaben dieser Projektarbeit gehörte, wird dieser Fehler nur hier erwähnt, sodass er bei einer zukünftigen Weiterentwicklung der Anwendung behoben werden kann.

Auch bei der Überarbeitung der Benutzeroberfläche bestehen weiterhin Bereiche, die stärker vereinheitlicht und strukturiert werden könnten. Die von dieser Projektarbeit vorgenommenen Änderungen konzentrierten sich nur auf die unmittelbar wichtigen Komponente, welche für die Bearbeitung der Projektarbeit relevant waren, sodass die Benutzeroberfläche nicht vollständig überarbeitet wurde. Eine vollständige Überarbeitung der Übersichtlichkeit und Struktur des Projekts hätte den vorgesehenen Umfang der Arbeit deutlich überschritten.

8.3. Gesamtbewertung

Insgesamt wie schon in den Kapitel 8.1 und 8.2 beschrieben, konnte die Projektarbeit trotz einiger Rückschläge erfolgreich umgesetzt werden. Die wichtigste Funktionalität wurde Erwartungsgemäß umgesetzt, man könnte sogar behaupten, dass die QR-Code-Funktionalität in einigen Bereichen sogar über die ursprünglichen Erwartungen überzeugt hat, da die Implementierung des Scanner durch die Debugmeldungen, recht zuverlässig und schnell im Vergleich zu meiner erwarteten Umsetzungszeit umgesetzt werden konnte.

Auch die folgenden Überarbeitungen haben nicht nur die Benutzeroberfläche verbessert, sondern auch gewissermaßen mehr Freude bei der Implementierung der Projektziele gebracht, trotz einiger Schwierigkeiten, bezüglich der Komponenten, welche nicht richtig umgesetzt wurden, die massiven Dateigrößen der verschiedenen Seiten und auch die nicht komplett realisierten Designideen, hat diese Projektarbeit in dieser Hinsicht gelehrt, dass man nicht mit einer übermäßigen Erwartungshaltung an die Umsetzung von Designideen herangehen sollte, sondern dass man sich schrittweise an die Umsetzung herantasten sollte, um die Umsetzung der Projektziele nicht zu gefährden.

Mit einer zusätzlichen Entwicklungszeit von ungefähr ein bis zwei Wochen würde ich den Schwerpunkt daher weniger auf weitere neue Funktionen legen, sondern vielmehr auf eine stärkere Strukturierung des bestehenden Projekts. Größere Komponenten könnten weiter aufgeteilt, Verantwortlichkeiten klarer voneinander getrennt und wiederkehrende Bestandteile noch stärker zentralisiert werden.

Wenn jedoch die Zeit um einen weiteren Monat verlängert werden würde, ohne eine erwartete größere Dokumentation zu schreiben, würde hier im Anschluss der Strukturierung des Projekts auch die Umsetzung von weiteren Designideen folgen, sodass diese Anwendung noch mehr die selbsterdachte Identität erhalten würde.

Trotz der beschriebenen Einschränkungen wurden die wesentlichen Ziele der Projektarbeit erreicht. Die bestehende Anwendung wurde sowohl funktional durch die QR-Code-Erweiterungen als auch gestalterisch und strukturell weiterentwickelt und bietet damit eine gute Grundlage für zukünftige Erweiterungen.

9. Zusammenfassung und Ausblick

9.1. Erreichte Ergebnisse

Im Rahmen dieser Projektarbeit wurde die bestehende Scavenger-Hunt-Webanwendung sowohl funktional als auch gestalterisch weiterentwickelt. Das zentrale Ziel der Arbeit war die Implementierung eines QR-Code-Scanners, welcher sowohl für den Beitritt zu einer Hunt als auch für die Antwortauswahl bei Aufgaben innerhalb einer Hunt verwendet werden kann.

Neben dem direkten Scan über die Kamera wurde die Funktionalität um weitere Möglichkeiten ergänzt, dazu gehören das Auslesen eines QR-Codes aus einer vorhandenen Bilddatei, der Wechsel zwischen mehreren Kameras sowie eine umfangreichere Behandlung möglicher Fehlerfälle.

Ein weiterer großer Bestandteil der Projektarbeit war die Überarbeitung der Benutzeroberfläche, um die zentralen Seiten der Anwendung hinsichtlich ihrer Gestaltung, Übersichtlichkeit und responsiven Darstellung zu verbessern. Zusätzlich wurden einheitlichere Farbkonzepte für den Light- und Dark-Mode hinzugefügt. Eine weitere Verbesserung war die teilweise Zentralisierung wiederkehrender Bestandteile des Frontends, um eine bessere Wartbarkeit zu gewährleisten.

Damit konnten die wesentlichen Ziele der Projektarbeit erreicht und die bestehende Scavenger-Hunt-Anwendung um neue Funktionen sowie eine verbesserte Benutzeroberfläche erweitert werden.

9.2. Ausblick

Die im Rahmen dieser Projektarbeit umgesetzten Erweiterungen bieten mehrere Ansatzpunkte für eine zukünftige Weiterentwicklung der Anwendung. Ein wesentlicher Bereich ist dabei die weitere Strukturierung des Frontends. Insbesondere größere Seiten könnten in verschiedene sinnvolle Komponenten aufgeteilt werden. Dadurch könnten Wartbarkeit und Übersichtlichkeit des Projekts weiter verbessert werden.

Auch eine Erweiterung der bisherigen Benutzeroberfläche bietet noch Potenzial

für zusätzliche Erweiterungen. Dazu gehören beispielsweise Mikroanimationen, zusätzliche Designelemente und eine noch stärkere visuelle Identität der Anwendung. Darüber hinaus könnte die Darstellung des downloadbaren QR-Codes weiter verbessert werden.

Eine zusätzlicher genannter Punkt der letzten Projektarbeit war die Erweiterung der Anwendung um eine AR-Funktionalität, welche die Nutzung von AR-Elementen innerhalb der Hunt ermöglichen würde. Diese Funktionalität wurde im Rahmen dieser Projektarbeit nicht in betracht gezogen, jedoch wirkt dies als ein möglicher Ansatzpunkt für eine zukünftige Weiterentwicklung der Anwendung, um die Interaktivität und den Spielspaß innerhalb der Hunt zu erhöhen.

Eine weitere mögliche Erweiterung der Anwendung könnte die Einführung von Gamification-Elementen sein und hinsichtlich auch dafür eine Freundesliste, sodass die Nutzer der Anwendung auch untereinander in Kontakt treten können und sich gegenseitig zu einer Hunt einladen können und zusätzlich durch die Gamification-Elemente einen Vergleich untereinander haben.

Weitere zukünftige Aufgaben könnten zu dem auch beinhalten, die Behebung der noch unbekannten Fehler innerhalb der Anwendung.

9.2.1. Erweiterbarkeit der Ergebnisse

Die entwickelte QR-Code-Funktionalität wurde so aufgebaut, dass sie nicht ausschließlich an einen einzelnen Anwendungsfall gebunden ist. Bereits innerhalb der Projektarbeit konnte dieselbe Scanner-Funktion sowohl für den Beitritt zu einer Hunt als auch für QR-Code-basierte Aufgaben verwendet werden.

Dadurch besteht die Möglichkeit, den Scanner zukünftig auch in weitere zukünftigen Funktionen der Anwendung zu integrieren. Denkbar wären beispielsweise zusätzliche Aufgabentypen, weitere Abläufe, die Teilnahme an einem Gruppen-Hunt, welchen man durch einen QR-Code beitreten kann, sowie die Nutzung von QR-Codes für die Freundschaftsverwaltung innerhalb der Anwendung.

Auch die eingeführten wiederverwendbaren UI-Komponenten und zentralen Gestaltungsdefinitionen können bei zukünftigen Erweiterungen genutzt werden und auch verbessert werden, dadurch können Seiten und Funktionen an der bestehenden Gestaltung stärker ausgerichtet werden, ohne wiederkehrende Elemente vollständig neu implementieren zu müssen.

9.2.2. Übertragbarkeit der Ergebnisse

Die grundlegenden Konzepte der entwickelten QR-Code-Funktionalität sind nicht ausschließlich auf die Scavenger-Hunt-Anwendung beschränkt. Die Kombination aus browserbasiertem Kamerazugriff, QR-Code-Erkennung und einer getrennten Verarbeitung des Scan-Ergebnisses kann grundsätzlich auch in anderen React-Webanwendungen eingesetzt werden. Dies betrifft insbesondere Anwendungen, bei denen QR-Codes zur Identifikation, Navigation oder Übertragung kurzer Informationen verwendet werden.

Zudem durch die Einarbeitung mit Hilfe der Debugmeldungen kann das grundlegende Konzept der schnellen und verlässlichen Erkennung von Fehlerfällen auch in anderen Anwendungen eingesetzt werden, sodass die Fehlerquellen schneller erkannt und behoben werden können.

Ein weiterer Aspekt der Übertragbarkeit betrifft die im Rahmen der UI/UX-Überarbeitung eingeführten wiederverwendbaren Komponenten und zentral definierten Gestaltungswerte, welche schon davor in einigen vorherigen Projekten vom Beginn aus eingesetzt wurden, jedoch durch diese Projektarbeit eine verstärkte Bedeutung erhalten haben, welche durch die Einarbeitung in eine größere Anwendung, welche zu dem ursprünglich einer anderen Projektgruppe entwickelt wurde.

10. Verwendete Technologien und Bibliotheken

Im Rahmen der Scavenger-Hunt-Anwendung werden verschiedene Technologien und Bibliotheken eingesetzt. Die grundlegenden Technologien wurden bereits aus dem vorherigen Projektstand übernommen und im Rahmen dieser Projektarbeit teilweise um weitere Bibliotheken erweitert.

10.1. Frontend

React

React bildet die Grundlage des Frontends und wird für den komponentenbasierten Aufbau der Benutzeroberfläche verwendet. Dabei werden unter anderem folgende Bibliotheken eingesetzt:

- `react-dom` – Darstellung der React-Anwendung im Browser
- `react-router-dom` – Navigation zwischen den verschiedenen Seiten
- `i18next` – Verwaltung der Übersetzungen
- `react-i18next` – Integration von i18next in React
- `react-icons` – Bereitstellung verschiedener Icons
- `leaflet` – Darstellung interaktiver Karten
- `react-leaflet` – Integration von Leaflet in React
- `@hello-pangea/dnd` – Drag-and-Drop-Funktionalität
- `react-qr-code` – Generierung der QR-Codes für QR-Code-basierte Aufgaben
- `jsQR` – Erkennung und Dekodierung von QR-Codes

10.2. Backend

FastAPI

Das Backend der Anwendung basiert auf FastAPI und stellt die benötigten REST-Schnittstellen für das Frontend bereit. Zu den verwendeten Bibliotheken gehören unter anderem:

- `fastapi-users` – Benutzerverwaltung und Authentifizierung
- `SQLAlchemy` – Datenbankzugriff und objektrelationale Abbildung
- `asyncpg` – Asynchrone PostgreSQL-Anbindung
- `psycopg2-binary` – PostgreSQL-Treiber
- `uvicorn` – ASGI-Server für die FastAPI-Anwendung

10.3. Datenbank

PostgreSQL

PostgreSQL wird für die persistente Speicherung der Anwendungsdaten verwendet.

10.4. Containerisierung und Bereitstellung

Docker und Docker Compose

Docker wird zur Containerisierung der einzelnen Bestandteile der Anwendung verwendet. Docker Compose ermöglicht die gemeinsame Konfiguration und Inbetriebnahme von Frontend, Backend und Datenbank.

Bibliothek des ersten QR-Code-Prototyps

Für die erste prototypische Umsetzung des QR-Code-Scanners wurde `html5-qrcode` eingesetzt. Diese Bibliothek wurde in der finalen Implementierung durch `jsQR` in Kombination mit der `MediaDevices` API ersetzt.

10.5. Relevante Dokumentationsbereiche

Für die Einarbeitung und Umsetzung der Erweiterungen wurden insbesondere die folgenden Bereiche der jeweiligen Dokumentationen verwendet:

- **React-Dokumentation:** Komponenten und deren Wiederverwendung, State-Verwaltung sowie die Hooks `useState`, `useEffect` und `useContext`. [13, 12]
- **MediaDevices API:** Kamerazugriff mit `getUserMedia()`, Berechtigungen und sichere Browserkontexte, das Ermitteln verfügbarer Kameras mit `enumerateDevices()`, die Kameraauswahl über `facingMode` beziehungsweise `deviceId` sowie das Beenden nicht mehr benötigter Media-Tracks. [6, 5, 8, 7]
- **jsQR:** Verarbeitung von Bilddaten, Verwendung mit Kamerastreams, Übergabe von `ImageData`, Rückgabewerte und `inversionAttempts`. [1]
- **html5-qrcode:** Grundlegende Einrichtung des Scanners, Kamerascan, Scan aus Bilddateien sowie die Klassen `Html5Qrcode` und `Html5QrcodeScanner`. [14]
- **QR-Code-Grundlagen:** Aufbau und Eigenschaften von QR-Codes, Positions-erkennung, Datenkapazität, Standardisierung und Fehlerkorrektur. [2]
- **Responsive Webdesign:** Responsive Layouts, Media Queries, Breakpoints und die Anpassung an unterschiedliche Viewport-Größen. [9]
- **CSS Custom Properties:** Zentrale Definition und Wiederverwendung von CSS-Variablen sowie deren Verwendung mit `var()` und `:root`. [10]
- **Canvas API und Scanverarbeitung:** Auslesen der Pixeldaten mit `getImageData()` sowie Steuerung des wiederkehrenden Scanvorgangs mit `requestAnimationFrame()`. [4, 11]
- **react-qrcode:** Generierung und Darstellung von QR-Codes innerhalb von React-Komponenten sowie Übergabe des zu codierenden Wertes. [3]

11. Einsatz von KI-Werkzeugen

Im Rahmen der Projektarbeit wurden verschiedene KI-Werkzeuge unterstützend eingesetzt.

Die KI-gestützte Ergänzungsfunktion in Visual Studio Code wurde hauptsächlich zur schnellen Vervollständigung kleinerer Codeabschnitte sowie zur Ergänzung und groben Rechtschreibkorrektur der Dokumentation verwendet. Die generierten Vorschläge wurden dabei überprüft und teilweise angepasst, da sie nicht immer der gewünschten Wortwahl oder Code-Struktur entsprachen.

ChatGPT wurde während der Vorbereitung als Recherche- und Einarbeitungswerkzeug verwendet, unter anderem zur Entwicklung einer strukturierten Vorgehensweise für die Implementierung des QR-Code-Scanners und zum besseren Verständnis des bestehenden Quellcodes. Während der Implementierung wurde das Modell hauptsächlich zur Fehlersuche und bei komplexeren Fehlern zur Erarbeitung möglicher Lösungsansätze eingesetzt. Dies diente insbesondere dazu, dass ich ein gewissen Überblick über die Funktionsweise der verwendeten Bibliotheken und deren Zusammenspiel mit der bestehenden Anwendung erhalten konnte, zu dem diente es auch als Hilfeleistung bei der Frage, wo ich mit der Implementierung beginnen sollte.

Bei der Dokumentation wurde ChatGPT zur Unterstützung bei der Strukturierung, zur Überprüfung des roten Fadens sowie zur groben sprachlichen Überarbeitung und Umformulierung einzelner Textabschnitte verwendet. Dies diente insbesondere dazu, dass die Einhaltung des roten Fadens und die Verständlichkeit der Dokumentation gewährleistet werden konnte, da ich während des Schreibens der Dokumentation oftmals den Überblick über die Gesamtstruktur verloren habe und oft mehrfach dieselben Inhalte in unterschiedlichen Kapiteln aber auch im gleichen Kapitel wiederholt habe.

Literatur

- [1] `cozmo.jsQR`. URL: <https://github.com/cozmo/jsQR> (besucht am 13.08.2026).
- [2] DENSO WAVE INCORPORATED. *What is a QR Code?* URL: <https://www.qrcode.com/en/about/index.html> (besucht am 06.08.2026).
- [3] Ross Khanas. *react-qr-code*. URL: <https://github.com/rosskhanas/react-qr-code> (besucht am 15.08.2026).
- [4] MDN Web Docs. *CanvasRenderingContext2D: getImageData() method*. URL: <https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/getImageData> (besucht am 14.08.2026).
- [5] MDN Web Docs. *MediaDevices: enumerateDevices() method*. URL: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/enumerateDevices> (besucht am 11.08.2026).
- [6] MDN Web Docs. *MediaDevices: getUserMedia() method*. URL: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia> (besucht am 10.08.2026).
- [7] MDN Web Docs. *MediaStreamTrack: stop() method*. URL: <https://developer.mozilla.org/en-US/docs/Web/API/MediaStreamTrack/stop> (besucht am 06.08.2026).
- [8] MDN Web Docs. *MediaTrackConstraints: facingMode property*. URL: <https://developer.mozilla.org/en-US/docs/Web/API/MediaTrackConstraints/facingMode> (besucht am 06.08.2026).
- [9] MDN Web Docs. *Responsive Web Design*. URL: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design (besucht am 14.08.2026).
- [10] MDN Web Docs. *Using CSS custom properties*. URL: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascading_variables/Using_custom_properties (besucht am 11.08.2026).
- [11] MDN Web Docs. *Window: requestAnimationFrame() method*. URL: <https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame> (besucht am 15.08.2026).
- [12] React. *Built-in React Hooks*. URL: <https://react.dev/reference/react/hooks> (besucht am 10.08.2026).
- [13] React. *Quick Start*. URL: <https://react.dev/learn> (besucht am 10.08.2026).

- [14] ScanApp. *html5-qr code Documentation*. URL: <https://github.com/scanapp-org/html5-qr code-docs> (besucht am 06.08.2026).

A. Anhang A

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Und nun folgt – ob man es glaubt oder nicht – der dritte Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Nach diesem vierten Absatz beginnen wir eine neue Zählung. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld.

Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

B. Anhang B

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Und nun folgt – ob man es glaubt oder nicht – der dritte Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Nach diesem vierten Absatz beginnen wir eine neue Zählung. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist

selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.